Complete Linux Network Commands Reference

Every Command, Every Flag, Every Option — With Examples & Explanations

Author: Compufreq

Table of Contents

PART 1 — CORE NETWORK MANAGEMENT

ip — Network Interface, Routing & Tunnel Management 3
ping — ICMP Connectivity Testing 21
traceroute — Trace the Route to a Host 25
tracepath — Route Tracing Without Root 27
mtr — Combined Traceroute + Ping 28

PART 2 — SOCKET & TRAFFIC ANALYSIS

ss — Socket Statistics 31
netstat — Network Statistics (Legacy) 36

PART 3 — DATA TRANSFER & DOWNLOAD

curl — Data Transfer Tool 38
wget — Non-Interactive Network Downloader 48

PART 4 — DNS & NAME RESOLUTION

dig — DNS Lookup Utility 56
nslookup — DNS Query Tool 63
host — Simple DNS Lookup 65

PART 5 — SCANNING & PACKET CAPTURE

nmap — Network Scanner 67
tcpdump — Packet Capture & Analysis 75

PART 6 — FIREWALLS & SECURITY

iptables — Firewall (Netfilter) 83
ufw — Uncomplicated Firewall 90
firewall-cmd — Firewalld CLI (RHEL/CentOS/Fedora) 92

PART 7 — REMOTE ACCESS & FILE TRANSFER

ssh — Secure Shell 93

scp — Secure Copy 99
sftp — Secure FTP 101
rsync — Remote File Synchronization 103

PART 8 — NETWORK SWISS ARMY KNIVES

nc / netcat — Network Swiss Army Knife 108

PART 9 — PERFORMANCE & HARDWARE

iperf3 — Network Performance Testing 110
ethtool — NIC Hardware Settings 113
nmcli — NetworkManager CLI 115

QUICK REFERENCE — COMMANDS 26–56

whois — Domain Registration Lookup 116
hostname / hostnamectl — Hostname Management 116
ipcalc — IP Address Calculator 116
lsof — List Open Network Sockets 116
iftop — Real-Time Bandwidth Monitor 116
vnstat — Traffic Monitor (Historical) 117
nethogs — Per-Process Bandwidth Monitor 117
tc — Traffic Control (QoS) 117
iw — Wireless Configuration (Modern) 118
socat — Multipurpose Relay 118
openssl s_client — SSL/TLS Testing 119
telnet — Connectivity Testing (Legacy) 119
dhclient — DHCP Client 119
resolvectl / systemd-resolve — DNS Resolution 120
tshark — Terminal Wireshark 120
ab — Apache HTTP Benchmarking 120
fuser — Identify Processes Using Ports 121
bridge — Bridge Management (Modern) 121

APPENDICES

System Network Configuration Files 122
Kernel Network Parameters (sysctl) 123

1. `ip` — Network Interface, Routing & Tunnel Management

The `ip` command (from the `iproute2` package) is the primary tool for managing network interfaces, addresses, routing, tunnels, and neighbor entries on modern Linux. It replaces the older `ifconfig`, `route`, `arp`, and `vconfig` commands.

General Syntax

```
ip [ OPTIONS ] OBJECT { COMMAND | help }
```

Global Options (Apply to All Subcommands)

Flag	Long Form	Description
`-c`	`-color`	Enable color output for readability
`-s`	`-stats`	Output more statistics; use `-s -s` for even more
`-d`	`-details`	Output more detailed information
`-h`	`-human`	Output human-readable statistics (with units like K, M, G)
`-j`	`-json`	Output in JSON format (machine-parseable)
`-p`	`-pretty`	Pretty-print JSON output (use with `-j`)
`-f FAMILY`	`-family FAMILY`	Protocol family: `inet` (IPv4), `inet6` (IPv6), `link` (layer 2), `bridge`, `mpls`
`-4`		Shorthand for `-f inet` (IPv4 only)
`-6`		Shorthand for `-f inet6` (IPv6 only)
`-B`		Shorthand for `-f bridge`
`-0`		Shorthand for `-f link` (layer 2)
`-o`	`-oneline`	Output each record on a single line (replacing newlines with `\`)
`-r`	`-resolve`	Resolve DNS names for IP addresses
`-n NETNS`	`-netns NETNS`	Switch to the specified network namespace before running the command
`-a NETNS`	`-all`	Execute command across all network namespaces
`-t`	`-timestamp`	Display timestamps when monitoring
`-br`	`-brief`	Print brief output (compact table format)
`-rc`	`-rcvbuf`	Set netlink socket receive buffer size
`-l`	`-loops COUNT`	Maximum number of loops for `ip addr flush`
`-b`	`-batch FILE`	Read commands from batch file

`ip link` — Manage Network Interfaces

`ip link show` — Display Interface Information

```
# Show all interfaces
ip link show

# Show a specific interface
ip link show dev eth0

# Show only UP interfaces
ip link show up

# Show only interfaces of a specific type
ip link show type bridge
ip link show type vlan
ip link show type veth
ip link show type bond

# Show with statistics
ip -s link show

# Show in brief format
ip -br link show

# Show in JSON format
ip -j link show | jq .

# Filter by group
ip link show group default
```

Output fields explained:

- `<BROADCAST,MULTICAST,UP,LOWER_UP>` — Interface flags showing capabilities and state
  - `BROADCAST` — Can send broadcast frames
  - `MULTICAST` — Supports multicast
  - `UP` — Admin-enabled (turned on)
  - `LOWER_UP` — Physical link detected (cable plugged in)
  - `NO-CARRIER` — No physical link detected
  - `PROMISC` — Promiscuous mode enabled
  - `LOOPBACK` — Loopback device
- `mtu 1500` — Maximum Transmission Unit (bytes)
- `qdisc fq_codel` — Queueing discipline (traffic scheduler)
- `state UP` — Operational state
- `mode DEFAULT` — Interface mode
- `group default` — Interface group
- `qlen 1000` — Transmit queue length
- `link/ether 00:11:22:33:44:55` — MAC address
- `brd ff:ff:ff:ff:ff:ff` — Broadcast address

`ip link set` — Modify Interface Properties

```
# Bring interface up
sudo ip link set eth0 up

# Bring interface down
sudo ip link set eth0 down

# Change the MTU (Maximum Transmission Unit)
sudo ip link set eth0 mtu 9000          # Jumbo frames

# Change the MAC address
sudo ip link set eth0 down
sudo ip link set eth0 address 00:11:22:33:44:66
sudo ip link set eth0 up

# Rename an interface (must be down first)
sudo ip link set eth0 down
sudo ip link set eth0 name lan0
sudo ip link set lan0 up

# Enable promiscuous mode (capture all traffic on the segment)
sudo ip link set eth0 promisc on

# Disable promiscuous mode
sudo ip link set eth0 promisc off

# Set transmit queue length
sudo ip link set eth0 txqueuelen 2000

# Set the interface to multicast
sudo ip link set eth0 multicast on

# Assign to a network namespace
sudo ip link set eth0 netns my_namespace

# Set interface master (attach to bridge/bond)
sudo ip link set eth0 master br0

# Remove from master
sudo ip link set eth0 nomaster

# Set interface group
sudo ip link set eth0 group 42

# Set ARP on/off
sudo ip link set eth0 arp off          # Disable ARP
sudo ip link set eth0 arp on           # Enable ARP

# Set dynamic flag
sudo ip link set eth0 dynamic on
```

All `ip link set` flags:

Flag	Description
`up` / `down`	Enable/disable the interface
`arp on/off`	Enable/disable ARP on the interface
`multicast on/off`	Enable/disable multicast
`promisc on/off`	Enable/disable promiscuous mode
`dynamic on/off`	Enable/disable dynamic flag
`allmulticast on/off`	Receive all multicast frames
`mtu MTU`	Set Maximum Transmission Unit
`name NAME`	Rename interface (must be down)
`address LLADDR`	Set MAC/hardware address
`broadcast LLADDR`	Set broadcast address
`peer LLADDR`	Set peer address (point-to-point)
`txqueuelen NUM`	Set transmit queue length
`netns NETNS`	Move to network namespace
`master DEVICE`	Attach to master device (bridge/bond)
`nomaster`	Detach from master device
`group GROUP`	Set interface group
`alias NAME`	Set interface alias string
`vf NUM`	Virtual Function settings (SR-IOV)
`xdp`	Set XDP (eXpress Data Path) program
`type TYPE ARGS`	Set type-specific attributes

`ip link add` — Create Virtual Interfaces

```
# Create a bridge
sudo ip link add br0 type bridge

# Create a VLAN interface (VLAN ID 100 on eth0)
sudo ip link add link eth0 name eth0.100 type vlan id 100

# Create a veth pair (virtual ethernet pair)
sudo ip link add veth0 type veth peer name veth1

# Create a bond
sudo ip link add bond0 type bond mode 802.3ad

# Create a dummy interface
sudo ip link add dummy0 type dummy

# Create a macvlan interface
sudo ip link add macvlan0 link eth0 type macvlan mode bridge

# Create a TUN device
sudo ip tuntap add dev tun0 mode tun

# Create a TAP device
sudo ip tuntap add dev tap0 mode tap

# Create a GRE tunnel
sudo ip link add gre1 type gre remote 203.0.113.1 local 198.51.100.1 ttl 255

# Create a VXLAN interface
sudo ip link add vxlan0 type vxlan id 42 remote 203.0.113.1 dstport 4789 dev eth0

# Create a WireGuard interface
sudo ip link add wg0 type wireguard

# Create a GENEVE tunnel
sudo ip link add geneve0 type geneve id 100 remote 10.0.0.1

# Create an IPIP tunnel
sudo ip link add ipip1 type ipip remote 203.0.113.1 local 198.51.100.1
```

Supported interface types:

Type	Description
`bridge`	Ethernet bridge
`bond`	Bonding (link aggregation)
`vlan`	802.1Q VLAN
`veth`	Virtual Ethernet pair
`dummy`	Dummy interface
`macvlan`	MAC-based VLAN
`macvtap`	MAC-based VLAN with TAP
`ipvlan`	IP-based VLAN
`vxlan`	Virtual eXtensible LAN
`gre` / `gretap`	GRE tunnel
`ip6gre` / `ip6gretap`	IPv6 GRE tunnel
`ipip`	IPv4 in IPv4 tunnel
`sit`	IPv6 in IPv4 tunnel
`ip6tnl`	IPv6 tunnel
`vti` / `vti6`	Virtual Tunnel Interface
`wireguard`	WireGuard VPN
`geneve`	GENEVE tunnel
`erspan`	ERSPAN tunnel
`xfrm`	Transform (IPsec)
`netdevsim`	Network device simulator
`nlmon`	Netlink monitor
`ifb`	Intermediate Functional Block
`can`	CAN bus

`ip link delete` — Remove Virtual Interfaces

```
sudo ip link delete br0
sudo ip link delete eth0.100
sudo ip link delete veth0          # Also deletes peer veth1
```

`ip addr` / `ip address` — Manage IP Addresses

`ip addr show` — Display IP Addresses

```
# Show all addresses
ip addr show
ip addr                              # Shorthand
ip a                                 # Even shorter

# Show for a specific interface
ip addr show dev eth0

# Show only IPv4 addresses
ip -4 addr show

# Show only IPv6 addresses
ip -6 addr show

# Show brief format
ip -br addr show

# Show with scope filter
ip addr show scope global            # Only globally routable addresses
ip addr show scope link              # Only link-local addresses
ip addr show scope host              # Only host addresses (loopback)

# Show only primary addresses
ip addr show primary

# Show only secondary addresses
ip addr show secondary

# Show addresses matching a label
ip addr show label "eth0:*"

# Show for a specific type
ip addr show type bridge

# Show temporary (privacy) IPv6 addresses
ip addr show temporary

# Show deprecated addresses
ip addr show deprecated

# Show tentative addresses (DAD in progress)
ip addr show tentative
```

Output fields explained:

- `inet 192.168.1.100/24` — IPv4 address with prefix length
- `brd 192.168.1.255` — Broadcast address
- `scope global` — Address scope (global, link, host)
- `dynamic` — Assigned by DHCP
- `noprefixroute` — No automatic prefix route
- `valid_lft forever` — Valid lifetime (or seconds remaining)
- `preferred_lft forever` — Preferred lifetime

`ip addr add` — Assign IP Addresses

```
# Add an IPv4 address
sudo ip addr add 192.168.1.100/24 dev eth0

# Add an IPv4 address with broadcast
sudo ip addr add 192.168.1.100/24 brd 192.168.1.255 dev eth0

# Add an IPv4 address with a label (alias)
sudo ip addr add 192.168.1.101/24 dev eth0 label eth0:1

# Add an IPv6 address
sudo ip addr add 2001:db8::1/64 dev eth0

# Add a point-to-point address
sudo ip addr add 10.0.0.1 peer 10.0.0.2/32 dev tun0

# Add with specific valid and preferred lifetimes (seconds)
sudo ip addr add 192.168.1.100/24 dev eth0 valid_lft 3600 preferred_lft 1800

# Prevent automatic prefix route creation
sudo ip addr add 192.168.1.100/24 dev eth0 noprefixroute

# Add without DAD (Duplicate Address Detection) for IPv6
sudo ip addr add 2001:db8::1/64 dev eth0 nodad

# Add with home flag (Mobile IPv6)
sudo ip addr add 2001:db8::1/64 dev eth0 home
```

`ip addr add` flags:

Flag	Description
`dev DEVICE`	Interface to add address to
`local ADDRESS`	The address (can omit `local` keyword)
`peer ADDRESS`	Peer address (point-to-point)
`broadcast ADDRESS`	Broadcast address (`+` for auto-calculate)
`label LABEL`	Interface label/alias (e.g., `eth0:1`)
`scope SCOPE`	`global`, `link`, `host`, or numeric
`valid_lft SECONDS`	Valid lifetime (`forever` or seconds)
`preferred_lft SECONDS`	Preferred lifetime
`noprefixroute`	Don't add prefix route automatically
`home`	Home address (Mobile IPv6)
`nodad`	Skip Duplicate Address Detection
`mngtmpaddr`	Manage temporary addresses

`ip addr del` — Remove IP Addresses

```
# Remove a specific address
sudo ip addr del 192.168.1.100/24 dev eth0

# Remove an IPv6 address
sudo ip addr del 2001:db8::1/64 dev eth0
```

`ip addr flush` — Remove All Addresses from an Interface

```
# Flush all addresses from an interface
sudo ip addr flush dev eth0

# Flush only IPv4 addresses
sudo ip -4 addr flush dev eth0

# Flush only link-local addresses
sudo ip addr flush dev eth0 scope link

# Flush with label filter
sudo ip addr flush label "eth0:*"
```

`ip route` — Manage the Routing Table

`ip route show` — Display Routes

```
# Show all routes (main table)
ip route show
ip route                              # Shorthand
ip r                                  # Even shorter

# Show only IPv6 routes
ip -6 route show

# Show a specific routing table
ip route show table local
ip route show table all               # All tables

# Show the route for a specific destination
ip route get 8.8.8.8                  # Shows which route would be used
ip route get 8.8.8.8 from 192.168.1.100 iif eth0

# Show routes matching a prefix
ip route show match 10.0.0.0/8

# Show routes exactly matching a prefix
ip route show exact 10.0.0.0/24

# Show routes via a specific gateway
ip route show via 192.168.1.1

# Show routes on a specific device
ip route show dev eth0

# Show routes with a specific protocol
ip route show proto static
ip route show proto dhcp
ip route show proto kernel
ip route show proto bird            # BGP/OSPF via BIRD

# Show cached routes
ip route show cache

# Show routes with a specific scope
ip route show scope link

# Show routes in a specific type
ip route show type local
ip route show type broadcast
ip route show type unreachable
```

Output fields explained:

- `default via 192.168.1.1 dev eth0` — Default route through gateway
- `proto kernel` — Route origin: `kernel` (auto), `boot` (startup), `static` (manual), `dhcp`
- `scope link` — Route scope
- `src 192.168.1.100` — Preferred source address
- `metric 100` — Route metric (priority; lower = preferred)
- `mtu 1500` — Path MTU
- `advmss 1460` — Advertised MSS

`ip route add` — Add Routes

```
# Add a default gateway
sudo ip route add default via 192.168.1.1 dev eth0

# Add a network route
sudo ip route add 10.0.0.0/8 via 192.168.1.1

# Add a network route via a specific device
sudo ip route add 10.0.0.0/8 via 192.168.1.1 dev eth0

# Add a route with a specific metric
sudo ip route add 10.0.0.0/8 via 192.168.1.1 metric 100

# Add a route with a specific source address
sudo ip route add 10.0.0.0/8 via 192.168.1.1 src 192.168.1.100

# Add a host route (to a single IP)
sudo ip route add 10.0.0.5/32 via 192.168.1.1

# Add a route with MTU
sudo ip route add 10.0.0.0/8 via 192.168.1.1 mtu 1400

# Add an unreachable route (packets are rejected with ICMP)
sudo ip route add unreachable 10.99.0.0/16

# Add a blackhole route (packets are silently dropped)
sudo ip route add blackhole 10.99.0.0/16

# Add a prohibit route (packets rejected with ICMP admin-prohibited)
sudo ip route add prohibit 10.99.0.0/16

# Add a throw route (no route to destination in this table)
sudo ip route add throw 10.99.0.0/16

# Add to a specific routing table
sudo ip route add 10.0.0.0/8 via 192.168.1.1 table 100

# Add a multipath (ECMP) route with weights
sudo ip route add 10.0.0.0/8 \
  nexthop via 192.168.1.1 weight 1 \
  nexthop via 192.168.2.1 weight 2

# Add route with specific protocol tag
sudo ip route add 10.0.0.0/8 via 192.168.1.1 proto static

# Add route with window/rtt/rttvar hints
sudo ip route add 10.0.0.0/8 via 192.168.1.1 window 65535 rtt 100ms

# Add route with congestion control
sudo ip route add 10.0.0.0/8 via 192.168.1.1 congctl bbr

# Add route with advanced MSS
sudo ip route add 10.0.0.0/8 via 192.168.1.1 advmss 1400

# Add route with realm
sudo ip route add 10.0.0.0/8 via 192.168.1.1 realm 5

# Add a route with onlink (gateway is directly connected even without matching subnet)
sudo ip route add 10.0.0.0/8 via 192.168.1.1 dev eth0 onlink

# Replace a route (add or update)
sudo ip route replace default via 192.168.1.1 dev eth0

# Change route attributes (route must exist)
sudo ip route change 10.0.0.0/8 via 192.168.1.1 mtu 1400
```

All `ip route add` parameters:

Parameter	Description
`via ADDRESS`	Next-hop gateway address
`dev DEVICE`	Output interface
`src ADDRESS`	Preferred source address
`metric NUM` / `preference NUM`	Route priority (lower = preferred)
`mtu NUM`	Path MTU
`mtu lock NUM`	Path MTU locked (don't do PMTUD)
`advmss NUM`	Advertised MSS for TCP connections
`table TABLE`	Routing table name or number
`proto PROTO`	Route protocol: `static`, `boot`, `kernel`, `dhcp`, etc.
`scope SCOPE`	`global`, `link`, `host`
`type TYPE`	`unicast`, `local`, `broadcast`, `multicast`, `throw`, `unreachable`, `prohibit`, `blackhole`, `nat`
`weight NUM`	Multipath weight
`onlink`	Gateway is directly connected
`nexthop`	Multi-hop specification
`window NUM`	TCP window hint
`rtt TIME`	Initial RTT estimate
`rttvar TIME`	Initial RTT variance
`ssthresh NUM`	Slow start threshold
`cwnd NUM`	Congestion window size
`initcwnd NUM`	Initial congestion window
`initrwnd NUM`	Initial receive window
`quickack BOOL`	Enable/disable quick ACK
`congctl NAME`	Congestion control algorithm
`features FEATURES`	Route features (`ecn`)
`realm REALM`	Routing realm
`expires SECONDS`	Route expiration time
`pref PREFERENCE`	IPv6 route preference: `low`, `medium`, `high`

`ip route del` — Delete Routes

```
# Delete a route
sudo ip route del 10.0.0.0/8 via 192.168.1.1

# Delete the default route
sudo ip route del default

# Delete from a specific table
sudo ip route del 10.0.0.0/8 table 100
```

`ip route flush` — Remove Multiple Routes

```
# Flush all routes via a specific gateway
sudo ip route flush via 192.168.1.1

# Flush all routes in a table
sudo ip route flush table 100

# Flush the routing cache
sudo ip route flush cache

# Flush routes matching a protocol
sudo ip route flush proto dhcp
```

`ip neigh` — Manage ARP/NDP Cache (Neighbor Table)

```
# Show the neighbor table (ARP for IPv4, NDP for IPv6)
ip neigh show
ip neigh                              # Shorthand
ip n                                  # Even shorter

# Show for a specific interface
ip neigh show dev eth0

# Show only reachable entries
ip neigh show nud reachable

# Show entries with a specific state
ip neigh show nud stale
ip neigh show nud failed
ip neigh show nud permanent

# Add a static ARP entry
sudo ip neigh add 192.168.1.50 lladdr 00:11:22:33:44:55 dev eth0

# Add a permanent ARP entry
sudo ip neigh add 192.168.1.50 lladdr 00:11:22:33:44:55 dev eth0 nud permanent

# Delete an entry
sudo ip neigh del 192.168.1.50 dev eth0

# Change an entry
sudo ip neigh change 192.168.1.50 lladdr 00:11:22:33:44:66 dev eth0

# Replace an entry (add or update)
sudo ip neigh replace 192.168.1.50 lladdr 00:11:22:33:44:55 dev eth0

# Flush the ARP cache
sudo ip neigh flush all
sudo ip neigh flush dev eth0
sudo ip neigh flush nud stale

# Show proxy ARP entries
ip neigh show proxy
```

NUD (Neighbor Unreachability Detection) states:

State	Description
`permanent`	Manually configured, never expires
`noarp`	Valid but no ARP needed (e.g., loopback)
`reachable`	Valid, recently confirmed reachable
`stale`	Valid but possibly unreachable
`delay`	Waiting to send probe
`probe`	Actively probing
`failed`	Resolution failed
`incomplete`	ARP request sent, waiting for reply
`none`	Pseudo state

`ip rule` — Policy-Based Routing Rules

```
# Show all routing rules
ip rule show
ip rule list

# Add a rule: traffic from 192.168.1.0/24 uses table 100
sudo ip rule add from 192.168.1.0/24 table 100

# Add a rule with priority
sudo ip rule add from 192.168.1.0/24 table 100 priority 100

# Route traffic to a specific destination through table 200
sudo ip rule add to 10.0.0.0/8 table 200

# Route based on firewall mark
sudo ip rule add fwmark 1 table 100

# Route based on incoming interface
sudo ip rule add iif eth0 table 100

# Route based on outgoing interface
sudo ip rule add oif eth1 table 200

# Route based on TOS (Type of Service)
sudo ip rule add tos 0x10 table 100

# Route based on IP protocol
sudo ip rule add ipproto tcp table 100

# Route based on source port range
sudo ip rule add sport 1024-65535 table 100

# Route based on destination port range
sudo ip rule add dport 80 table 100

# Add an unreachable rule
sudo ip rule add from 10.99.0.0/16 unreachable

# Delete a rule
sudo ip rule del from 192.168.1.0/24 table 100

# Flush all rules (DANGEROUS - removes all policy rules)
sudo ip rule flush
```

`ip tunnel` — Manage IP Tunnels

```
# Show all tunnels
ip tunnel show

# Create an IPIP tunnel
sudo ip tunnel add tun0 mode ipip remote 203.0.113.1 local 198.51.100.1

# Create a GRE tunnel
sudo ip tunnel add gre1 mode gre remote 203.0.113.1 local 198.51.100.1 ttl 255

# Create a SIT tunnel (IPv6 over IPv4)
sudo ip tunnel add sit1 mode sit remote 203.0.113.1 local 198.51.100.1

# Change tunnel parameters
sudo ip tunnel change gre1 ttl 128

# Delete a tunnel
sudo ip tunnel del tun0
```

Tunnel modes:

Mode	Description
`ipip`	IPv4 in IPv4
`gre`	Generic Routing Encapsulation
`sit`	IPv6 in IPv4 (Simple Internet Transition)
`isatap`	Intra-Site Automatic Tunnel Addressing Protocol
`vti`	Virtual Tunnel Interface (for IPsec)
`ip6ip6`	IPv6 in IPv6
`ipip6`	IPv4 in IPv6
`ip6gre`	GRE over IPv6
`any`	Any encapsulation

`ip netns` — Manage Network Namespaces

```
# List all named network namespaces
ip netns list

# Create a new namespace
sudo ip netns add my_ns

# Delete a namespace
sudo ip netns delete my_ns

# Execute a command in a namespace
sudo ip netns exec my_ns ip addr show
sudo ip netns exec my_ns bash        # Open a shell in the namespace

# Identify the namespace of a process
ip netns identify PID

# Attach a process to a namespace
ip netns attach my_ns PID

# Monitor namespace events
ip netns monitor

# Set a namespace as current (for subsequent ip commands)
sudo ip -n my_ns addr show           # -n shorthand for -netns
```

`ip maddr` — Manage Multicast Addresses

```
# Show all multicast addresses
ip maddr show

# Show for a specific interface
ip maddr show dev eth0

# Add a multicast address
sudo ip maddr add 01:00:5e:00:00:01 dev eth0

# Delete a multicast address
sudo ip maddr del 01:00:5e:00:00:01 dev eth0
```

`ip mroute` — Multicast Routing Cache

```
# Show multicast routing cache
ip mroute show
```

`ip monitor` — Real-Time Monitoring

```
# Monitor all changes (addresses, routes, links, etc.)
ip monitor all

# Monitor only link changes
ip monitor link

# Monitor only address changes
ip monitor address

# Monitor only route changes
ip monitor route

# Monitor only neighbor changes
ip monitor neigh

# Monitor with timestamps
ip -t monitor all

# Monitor in a specific namespace
sudo ip -n my_ns monitor all
```

`ip xfrm` — IPsec / Security Association Management

```
# Show IPsec Security Associations
ip xfrm state list

# Show IPsec Security Policies
ip xfrm policy list

# Monitor IPsec events
ip xfrm monitor

# Flush all SAs
sudo ip xfrm state flush

# Flush all policies
sudo ip xfrm policy flush
```

`ip tcp_metrics` — TCP Metrics Cache

```
# Show cached TCP metrics
ip tcp_metrics show

# Show for a specific destination
ip tcp_metrics show 8.8.8.8

# Flush cached metrics
sudo ip tcp_metrics flush

# Delete metrics for a specific destination
sudo ip tcp_metrics delete 8.8.8.8
```

`ip token` — IPv6 Tokenized Interface Identifiers

```
# Show current token
ip token show

# Set a token for an interface
sudo ip token set ::1234:5678:90ab:cdef dev eth0
```

`ip l2tp` — L2TPv3 Management

```
# Show L2TP tunnels
ip l2tp show tunnel

# Show L2TP sessions
ip l2tp show session

# Add a tunnel
sudo ip l2tp add tunnel tunnel_id 1 peer_tunnel_id 1 encap udp \
  local 198.51.100.1 remote 203.0.113.1 udp_sport 5000 udp_dport 5000

# Add a session
sudo ip l2tp add session tunnel_id 1 session_id 1 peer_session_id 1

# Delete a session
sudo ip l2tp del session tunnel_id 1 session_id 1

# Delete a tunnel
sudo ip l2tp del tunnel tunnel_id 1
```

2. `ping` — ICMP Connectivity Testing

Sends ICMP Echo Request packets to test whether a host is reachable and measure round-trip time.

Syntax

```
ping [OPTIONS] DESTINATION
```

All Flags and Options

Flag	Description
`-c COUNT`	Stop after sending COUNT packets
`-i INTERVAL`	Seconds between packets (default: 1). Intervals < 0.2 require root
`-w DEADLINE`	Timeout in seconds; ping exits after this many seconds regardless of packets sent/received
`-W TIMEOUT`	Time to wait for each response in seconds
`-s SIZE`	Payload size in bytes (default: 56, total ICMP = 64 with header)
`-t TTL`	Set the IP Time To Live
`-I INTERFACE`	Bind to a specific interface or source IP address
`-f`	Flood ping — sends packets as fast as possible (root only). Prints `.` for sent and backspace for received
`-l PRELOAD`	Send PRELOAD packets before waiting for replies (root only)
`-n`	Numeric output only (don't resolve hostnames)
`-q`	Quiet mode — only show summary at the end
`-v`	Verbose output
`-a`	Audible ping — beep on each reply
`-A`	Adaptive ping — adjust interval to RTT
`-b`	Allow pinging a broadcast address
`-B`	Do not allow ping to change source address
`-d`	Set SO_DEBUG socket option
`-D`	Print timestamp (Unix time + microseconds) before each line
`-F FLOW`	Set IPv6 flow label (IPv6 only)
`-L`	Suppress loopback of multicast packets
`-m MARK`	Set the routing mark on outgoing packets
`-M HINT`	Path MTU discovery strategy: `do` (set DF), `want` (try DF), `dont` (don't set DF)
`-N`	Use ICMP Node Information Queries (IPv6)
`-O`	Report outstanding ICMP ECHO replies before sending next
`-p PATTERN`	Fill padding bytes with hex pattern (e.g., `-p ff`)
`-Q TOS`	Set Quality of Service / DSCP bits in IP header
`-r`	Bypass routing table (send directly on attached network)
`-R`	Record route (IPv4; limited to 9 hops)
`-S SNDBUF`	Set socket send buffer size
`-T OPTION`	Set special IP timestamp options: `tsonly`, `tsandaddr`, `tsprespec`
`-U`	Print full user-to-user latency
`-4`	Force IPv4
`-6`	Force IPv6

Examples

```
# Basic ping
ping google.com
# Output: 64 bytes from 142.250.80.46: icmp_seq=1 ttl=118 time=11.2 ms

# Ping 5 times then stop
ping -c 5 192.168.1.1

# Ping with 0.2 second interval
ping -i 0.2 192.168.1.1

# Ping with a 10-second deadline (exit after 10s)
ping -w 10 192.168.1.1

# Ping with large packet size (test MTU)
ping -s 1472 -M do 192.168.1.1
# If packet is too large, you'll see: "Frag needed and DF set"
# This helps find the maximum MTU on the path

# Flood ping (root only, test stress/packet loss)
sudo ping -f -c 1000 192.168.1.1

# Ping from a specific source IP
ping -I 192.168.1.100 8.8.8.8

# Ping from a specific interface
ping -I eth0 8.8.8.8

# Ping with timestamps
ping -D -c 3 google.com

# Quiet mode (summary only)
ping -q -c 10 google.com
# Output:
# 10 packets transmitted, 10 received, 0% packet loss, time 9012ms
# rtt min/avg/max/mdev = 10.123/11.456/14.789/1.234 ms

# Ping with audible beep
ping -a google.com

# Set TTL to 10 (will fail if destination is >10 hops away)
ping -t 10 google.com

# Ping a broadcast address (discover hosts on LAN)
ping -b 192.168.1.255

# Numeric output (skip DNS resolution)
ping -n -c 3 8.8.8.8

# Adaptive ping (adjusts interval to match RTT)
ping -A -c 20 google.com

# Record route option
ping -R -c 1 google.com

# Set TOS/DSCP
ping -Q 0x10 google.com

# IPv6 ping
ping -6 ipv6.google.com
ping6 ipv6.google.com               # Alternative command
```

Understanding ping output:

```
PING google.com (142.250.80.46) 56(84) bytes of data.
64 bytes from lax17s55-in-f14.1e100.net (142.250.80.46): icmp_seq=1 ttl=118 time=11.2 ms
```

- `56(84)` — 56 bytes payload + 8 bytes ICMP header + 20 bytes IP header = 84 bytes total
- `icmp_seq=1` — Sequence number (gaps indicate packet loss)
- `ttl=118` — Remaining hops (started at 128, so ~10 hops away)
- `time=11.2 ms` — Round-trip time

Summary statistics:

```
--- google.com ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4006ms
rtt min/avg/max/mdev = 10.123/11.456/14.789/1.234 ms
```

- `min` — Fastest round trip
- `avg` — Average round trip
- `max` — Slowest round trip
- `mdev` — Standard deviation (jitter)

---

3. `traceroute` — Trace the Route to a Host

Maps every router (hop) between your machine and a destination by sending packets with incrementally increasing TTL values. When TTL expires at a router, it sends back an ICMP Time Exceeded message.

Syntax

```
traceroute [OPTIONS] DESTINATION [PACKET_LENGTH]
```

All Flags and Options

Flag	Description
`-4`	Force IPv4
`-6`	Force IPv6
`-I`	Use ICMP ECHO instead of UDP (root required)
`-T`	Use TCP SYN instead of UDP (root required; good for firewalled hosts)
`-U`	Use UDP (default)
`-d`	Enable socket debugging
`-F`	Set Don't Fragment flag
`-f FIRST_TTL`	Start with this TTL (default: 1)
`-g GATEWAY`	Use loose source routing through this gateway
`-i INTERFACE`	Bind to a specific interface
`-m MAX_TTL`	Maximum TTL / number of hops (default: 30)
`-n`	Numeric output only (don't resolve hostnames)
`-p PORT`	Destination port (default: 33434 for UDP, 80 for TCP)
`-q NQUERIES`	Number of probes per hop (default: 3)
`-r`	Bypass routing table
`-s SOURCE`	Use this source address
`-t TOS`	Set Type of Service / DSCP
`-w WAITTIME`	Seconds to wait for response per probe (default: 5)
`-z SENDWAIT`	Minimum time interval between probes in ms (default: 0)
`-A`	Perform AS path lookups (show ASN for each hop)
`-e`	Display ICMP extensions (MPLS labels, etc.)
`-M METHOD`	Method: `default`, `icmp`, `tcp`, `udp`, `udplite`, `dccp`, `raw`
`-N SQUERIES`	Simultaneous probes (default: 16)
`-O OPTION`	Module-specific option
`--mtu`	Discover and display path MTU
`--back`	Display backward route TTL
`--sport PORT`	Source port

Examples

```
# Basic traceroute
traceroute google.com
# Output:
#  1  gateway (192.168.1.1)  0.543 ms  0.397 ms  0.456 ms
#  2  isp-router (10.0.0.1)  5.234 ms  5.178 ms  5.312 ms
#  3  * * *                                                    # No response (firewall)
#  4  lax17s55-in-f14.1e100.net (142.250.80.46)  11.234 ms  11.456 ms  11.123 ms

# Use ICMP (better success through some firewalls)
sudo traceroute -I google.com

# Use TCP on port 443 (best for firewalled destinations)
sudo traceroute -T -p 443 google.com

# Numeric only (faster, no DNS resolution delay)
traceroute -n google.com

# Limit to 15 hops
traceroute -m 15 google.com

# Start from hop 5 (skip known initial hops)
traceroute -f 5 google.com

# Only 1 probe per hop (faster but less reliable)
traceroute -q 1 google.com

# Show AS numbers
traceroute -A google.com

# Use a specific source address
traceroute -s 192.168.1.100 google.com

# Discover path MTU
traceroute --mtu google.com

# Increase wait time per probe to 10 seconds
traceroute -w 10 google.com

# Use ICMP with Don't Fragment flag
sudo traceroute -I -F google.com

# Traceroute from a specific interface
traceroute -i eth0 google.com
```

Understanding the output:

Each line represents one hop. `*` means no response was received for that probe (the router may be configured to not respond, or a firewall is blocking).

4. `tracepath` — Route Tracing Without Root

Similar to `traceroute` but does not require root privileges. Also discovers the Path MTU.

Syntax

```
tracepath [OPTIONS] DESTINATION[/PORT]
```

All Flags

Flag	Description
`-4`	Force IPv4
`-6`	Force IPv6
`-n`	Numeric output (don't resolve names)
`-b`	Print both hostname and IP
`-l LENGTH`	Initial packet length (default: 65535 for IPv4, 128000 for IPv6)
`-m MAX_HOPS`	Maximum hops (default: 30)
`-p PORT`	Destination port

Examples

```
# Basic tracepath
tracepath google.com

# Tracepath to a specific port
tracepath google.com/443

# Numeric output
tracepath -n google.com

# Set max hops
tracepath -m 20 google.com

# IPv6
tracepath -6 ipv6.google.com
```

5. `mtr` — Combined Traceroute + Ping

`mtr` combines `traceroute` and `ping` into a single real-time diagnostic tool. It continuously probes each hop and updates statistics live.

Syntax

```
mtr [OPTIONS] DESTINATION
```

All Flags and Options

Flag	Description
`-r` / `--report`	Report mode — run then print final report (non-interactive)
`-w` / `--report-wide`	Wide report (don't truncate hostnames)
`-c COUNT`	Number of pings per hop (default: 10 in report mode, unlimited in interactive)
`-s SIZE`	Packet size in bytes
`-n` / `--no-dns`	Numeric output (no DNS resolution)
`-b` / `--show-ips`	Show both hostnames and IPs
`-o FIELDS`	Select output fields (see below)
`-i INTERVAL`	Interval between probes in seconds (default: 1)
`-m MAX_TTL`	Maximum TTL (default: 30)
`-f FIRST_TTL`	Starting TTL (default: 1)
`-4`	Force IPv4
`-6`	Force IPv6
`-u`	Use UDP instead of ICMP
`-T`	Use TCP SYN
`-P PORT`	Target port (for TCP/UDP)
`-L LOCALPORT`	Source port
`-e`	Display ICMP extensions
`-a ADDRESS`	Bind to source address
`-I INTERFACE`	Bind to interface
`-M MARK`	Set routing mark
`-Q TOS`	Set TOS/DSCP
`-z`	Show AS numbers
`-y NUM`	Show AS number in specific format: 0=none, 1=number, 2=name, 3=both
`--csv`	Output in CSV format
`--raw`	Output in raw format
`--xml`	Output in XML format
`--json`	Output in JSON format
`--gtk`	Use GTK interface
`--curses`	Use ncurses interface (default)
`--split`	Split output format
`--no-dns`	Same as `-n`

Output fields (`-o` flag):

Code	Field
`L`	Loss ratio
`D`	Dropped packets
`R`	Received packets
`S`	Sent packets
`N`	Newest RTT (ms)
`B`	Min/Best RTT (ms)
`A`	Average RTT (ms)
`W`	Max/Worst RTT (ms)
`V`	Standard deviation
`G`	Geometric mean
`J`	Current jitter
`M`	Jitter mean/avg
`X`	Worst jitter
`I`	Inter-arrival jitter

Examples

```
# Interactive mode (real-time updates)
mtr google.com

# Report mode (run then print summary)
mtr -r -c 100 google.com

# Wide report with 50 probes
mtr -rw -c 50 google.com

# Show IPs and hostnames
mtr -b google.com

# Use TCP on port 443
mtr -T -P 443 google.com

# JSON output for scripting
mtr --json -c 10 google.com

# Show AS numbers
mtr -z google.com

# Custom output fields (loss, sent, received, best, avg, worst, stdev)
mtr -o "LSRBAWV" google.com

# Numeric only with 0.5s interval
mtr -n -i 0.5 google.com

# Start from hop 3, max 20 hops
mtr -f 3 -m 20 google.com

# CSV output for later analysis
mtr --csv -c 100 google.com > mtr_report.csv

# Bind to specific source
mtr -a 192.168.1.100 google.com
```

6. `ss` — Socket Statistics

`ss` is the modern replacement for `netstat`. It retrieves socket information directly from the kernel and is significantly faster.

Syntax

```
ss [OPTIONS] [FILTER]
```

All Flags and Options

Flag	Description
`-t`	Show TCP sockets
`-u`	Show UDP sockets
`-w`	Show RAW sockets
`-x`	Show Unix domain sockets
`-d`	Show DCCP sockets
`-S`	Show SCTP sockets
`-l`	Show only listening sockets
`-a`	Show all sockets (listening + non-listening)
`-n`	Numeric output (don't resolve service names)
`-r`	Resolve IP addresses to hostnames
`-p`	Show process using the socket
`-e`	Show detailed socket info (uid, inode, cookie)
`-m`	Show socket memory usage
`-i`	Show TCP internal information (congestion control, RTT, etc.)
`-K`	Force kill sockets matching the filter (root required)
`-s`	Print socket statistics summary
`-o`	Show timer information
`-E`	Continuously display socket events
`-Z`	Display SELinux security context
`-z`	Display socket context (similar to `-Z`)
`-N NS-NAME`	Switch to network namespace
`-b`	Show BPF filter socket option
`-4`	Show IPv4 sockets only
`-6`	Show IPv6 sockets only
`-0`	Show PACKET sockets
`-f FAM-ILY`	Filter by address family: `unix`, `inet`, `inet6`, `link`, `netlink`, `vsock`, `tipc`, `xdp`
`-A QUERY`	Socket tables to dump: `all`, `inet`, `tcp`, `udp`, `raw`, `unix`, `packet`, `netlink`, `unix_dgram`, `unix_stream`, `unix_seqpacket`, `packet_raw`, `packet_dgram`, `dccp`, `sctp`, `vsock_stream`, `vsock_dgram`, `xdp`
`-D FILE`	Dump raw TCP socket data to FILE
`-F FILE`	Read filter from FILE
`-H`	Suppress header line
`--no-header`	Same as `-H`
`--tos`	Show TOS value
`--cgroup`	Show cgroup
`--tip-cinfo`	Show TIPC socket info
`-V`	Show version

State Filters

```
# Filter by connection state
ss state ESTABLISHED
ss state SYN-SENT
ss state SYN-RECV
ss state FIN-WAIT-1
ss state FIN-WAIT-2
ss state TIME-WAIT
ss state CLOSE-WAIT
ss state LAST-ACK
ss state CLOSING
ss state CLOSED
ss state LISTEN
ss state ALL

# Exclude a state
ss exclude LISTEN
ss exclude TIME-WAIT

# Combined state groups
ss state connected           # All except LISTEN and CLOSED
ss state synchronized        # ESTABLISHED + derivatives
ss state bucket              # TIME-WAIT + SYN-RECV
ss state big                 # Everything except bucket
```

Address/Port Filters

```
# Filter by source port
ss sport = :80
ss sport = :http
ss sport gt :1024
ss sport lt :1024
ss sport != :22

# Filter by destination port
ss dport = :443
ss dport = :https

# Filter by source address
ss src 192.168.1.100
ss src 192.168.1.0/24

# Filter by destination address
ss dst 8.8.8.8
ss dst 10.0.0.0/8

# Combine filters with 'and' / 'or'
ss -tn 'src 192.168.1.0/24 and dport = :443'
ss -tn '( dport = :80 or dport = :443 ) and src 192.168.1.0/24'

# Filter operators
# =  eq   : Equal
# != ne   : Not equal
# >  gt   : Greater than
# <  lt   : Less than
# >= ge   : Greater than or equal
# <= le   : Less than or equal
```

Examples

```
# Show all listening TCP sockets with process names
ss -tlnp
# Output:
# State   Recv-Q  Send-Q  Local Address:Port  Peer Address:Port  Process
# LISTEN  0       128     0.0.0.0:22          0.0.0.0:*          users:(("sshd",pid=1234,fd=3))
# LISTEN  0       511     0.0.0.0:80          0.0.0.0:*          users:(("nginx",pid=5678,fd=8))

# Show all established connections
ss -tn state established

# Show all sockets with timer info
ss -to

# Show TCP internal info (congestion window, RTT, etc.)
ss -ti

# Show socket memory usage
ss -tm

# Show summary
ss -s
# Output:
# Total: 356
# TCP:   15 (estab 3, closed 0, orphaned 0, timewait 0)
# Transport    Total     IP        IPv6
# RAW          1         0         1
# UDP          8         5         3
# TCP          15        10        5
# INET         24        15        9
# FRAG         0         0         0

# Show UDP sockets
ss -ulnp

# Show Unix domain sockets
ss -x

# Show all sockets on port 443
ss -tn dport = :443

# Show connections from a specific subnet
ss -tn src 10.0.0.0/8

# Kill all TIME-WAIT sockets to port 80
sudo ss -K dport = :80 state time-wait

# Monitor socket events in real time
ss -E

# Show sockets in JSON format (via filter to jq)
ss -tn | head

# Show SELinux context
ss -Ztn

# Show all SCTP sockets
ss -S

# Show detailed info (uid, inode)
ss -teln
```

7. `netstat` — Network Statistics (Legacy)

`netstat` is the older tool for displaying connections, routing tables, and interface statistics. Replaced by `ss` and `ip` but still commonly used.

Syntax

```
netstat [OPTIONS]
```

All Flags and Options

Flag	Description
`-t`	Show TCP connections
`-u`	Show UDP connections
`-l`	Show only listening sockets
`-a`	Show all sockets (listening + established)
`-n`	Numeric addresses and ports (don't resolve)
`-p`	Show PID/program name (root required for all processes)
`-r`	Show routing table
`-i`	Show interface statistics
`-g`	Show multicast group memberships
`-s`	Show per-protocol statistics
`-c`	Continuous mode (update every second)
`-e`	Show additional info (user, inode)
`-o`	Show timers
`-w`	Show RAW sockets
`-x`	Show Unix sockets
`-W`	Don't truncate IP addresses
`-v`	Verbose
`--numeric-hosts`	Don't resolve hostnames
`--numeric-ports`	Don't resolve port names
`--numeric-users`	Don't resolve usernames
`-A FAMILY`	Address families: `inet`, `inet6`, `unix`, `ipx`, `ax25`, `netrom`, `econet`, `ddp`, `bluetooth`
`-C`	Print routing info from route cache
`-F`	Print routing info from FIB
`-M`	Show masqueraded connections
`-Z`	Show SELinux security context

Examples

```
# Show all listening TCP/UDP ports with process names
netstat -tulnp

# Show all connections (listening + established)
netstat -an

# Show only TCP connections
netstat -tn

# Show routing table
netstat -rn

# Show interface statistics
netstat -i

# Show extended interface statistics
netstat -ie               # Similar to ifconfig

# Show per-protocol statistics (TCP/UDP/ICMP)
netstat -s

# Show TCP statistics only
netstat -st

# Show multicast group memberships
netstat -g

# Continuous monitoring
netstat -c -tn

# Show all Unix sockets
netstat -x

# Show timers
netstat -to

# Wide output (don't truncate addresses)
netstat -Wtn

# Show with SELinux context
netstat -Ztn

# Count connections by state
netstat -tn | awk '{print $6}' | sort | uniq -c | sort -rn

# Show masqueraded connections
netstat -M
```

8. `curl` — Data Transfer Tool

`curl` transfers data to or from a server using URLs. Supports HTTP, HTTPS, FTP, FTPS, SCP, SFTP, TFTP, LDAP, TELNET, MQTT, and many more protocols.

Syntax

```
curl [OPTIONS] URL [URL...]
```

All Major Flags and Options

HTTP Method and Request Control

Flag	Description
`-X METHOD` / `--request METHOD`	Set HTTP method: `GET`, `POST`, `PUT`, `DELETE`, `PATCH`, `HEAD`, `OPTIONS`
`-d DATA` / `--data DATA`	Send POST data (implies `-X POST`). Use `@filename` to read from file
`--data-raw DATA`	Send POST data without interpreting `@`
`--data-urlencode DATA`	URL-encode the data before sending
`--data-binary DATA`	Send binary data with no processing
`--data-ascii DATA`	Same as `--data`
`-F "field=value"` / `--form`	Multipart form data (file upload). Use `@filename` for files
`-G` / `--get`	Force GET method even with `-d` data (appends to URL as query params)
`-I` / `--head`	Send HEAD request (headers only)

Headers and Authentication

Flag	Description
`-H "Header: Value"` / `--header`	Add/replace HTTP header. Use `-H "Header:"` to remove
`-A "Agent"` / `--user-agent`	Set User-Agent header
`-e URL` / `--referer`	Set Referer header
`-u user:pass` / `--user`	HTTP Basic authentication
`--basic`	Use HTTP Basic auth (default)
`--digest`	Use HTTP Digest auth
`--ntlm`	Use HTTP NTLM auth
`--negotiate`	Use HTTP Negotiate (SPNEGO/Kerberos) auth
`--anyauth`	Auto-select best auth method
`-b "cookies"` / `--cookie`	Send cookies (string or `@file`)
`-c FILE` / `--cookie-jar`	Save cookies to file
`-j` / `--junk-session-cookies`	Discard session cookies from cookie jar
`--oauth2-bearer TOKEN`	OAuth 2.0 Bearer token
`--aws-sigv4 PROVIDER`	AWS Signature Version 4 signing

Output and Verbosity

Flag	Description
`-o FILE` / `--output`	Write output to FILE
`-O` / `--remote-name`	Save with remote filename
`-J` / `--remote-header-name`	Use Content-Disposition filename
`-s` / `--silent`	Silent mode (no progress, no errors)
`-S` / `--show-error`	Show errors even in silent mode
`-v` / `--verbose`	Verbose output (shows request/response headers)
`--trace FILE`	Full hex trace to FILE
`--trace-ascii FILE`	Full ASCII trace to FILE
`--trace-time`	Add timestamps to trace
`-w FORMAT` / `--write-out`	Print info after transfer (see format variables below)
`-i` / `--include`	Include response headers in output
`-D FILE` / `--dump-header`	Save response headers to FILE
`--no-progress-meter`	Hide progress bar but still show output and errors
`--progress-bar`	Simple progress bar instead of statistics
`-#`	Same as `--progress-bar`

Redirects and Retries

Flag	Description
`-L` / `--location`	Follow redirects
`--max-redirs NUM`	Maximum number of redirects (default: 50)
`--post301`	Keep POST method after 301 redirect
`--post302`	Keep POST method after 302 redirect
`--post303`	Keep POST method after 303 redirect
`--retry NUM`	Retry on transient failures
`--retry-delay SECONDS`	Wait between retries
`--retry-max-time SECONDS`	Maximum total retry time
`--retry-all-errors`	Retry on all errors (not just transient)

SSL/TLS

Flag	Description
`-k` / `--insecure`	Skip SSL certificate verification
`--cacert FILE`	Use this CA certificate bundle
`--capath DIR`	CA certificate directory
`--cert FILE`	Client certificate
`--cert-type TYPE`	Certificate type: `PEM`, `DER`, `ENG`
`--key FILE`	Private key
`--key-type TYPE`	Key type: `PEM`, `DER`, `ENG`
`--pass PHRASE`	Private key passphrase
`--ciphers LIST`	SSL ciphers to use
`--tls-max VERSION`	Maximum TLS version: `1.0`, `1.1`, `1.2`, `1.3`
`--tlsv1`	Force TLS 1.x
`--tlsv1.0` / `--tlsv1.1` / `--tlsv1.2` / `--tlsv1.3`	Force specific TLS version
`--ssl`	Try SSL/TLS
`--ssl-reqd`	Require SSL/TLS
`--crl-file FILE`	Certificate Revocation List
`--cert-status`	Verify server certificate via OCSP
`--pinnedpubkey FILE`	Pin public key (file or hash)
`--ssl-allow-beast`	Allow BEAST vulnerability
`--ssl-no-revoke`	Disable cert revocation checks (Windows)
`--ssl-revoke-best-effort`	Best-effort revocation check

Connection Control

Flag	Description
`--connect-timeout SECONDS`	Maximum time for connection phase
`-m SECONDS` / `--max-time`	Maximum total transfer time
`--speed-limit BYTES`	Minimum speed in bytes/sec before aborting
`--speed-time SECONDS`	Time to measure speed-limit
`--limit-rate RATE`	Limit transfer speed (e.g., `100K`, `1M`)
`--keepalive-time SECONDS`	TCP keepalive interval
`--no-keepalive`	Disable TCP keepalive
`--interface IFACE`	Use this interface
`--local-port RANGE`	Local port range
`--dns-servers SERVERS`	Use custom DNS servers
`--resolve HOST:PORT:ADDR`	Custom DNS resolution
`--connect-to HOST:PORT:CONNECT_HOST:CONNECT_PORT`	Connect to alternative host
`--happy-eyeballs-timeout-ms MS`	Happy Eyeballs timeout
`--tcp-nodelay`	Enable TCP_NODELAY
`--tcp-fastopen`	Enable TCP Fast Open

Proxy

Flag	Description
`-x PROXY` / `--proxy`	Use proxy `[protocol://]host[:port]`
`--proxy-user user:pass`	Proxy authentication
`--proxy-basic`	Proxy Basic auth
`--proxy-digest`	Proxy Digest auth
`--proxy-ntlm`	Proxy NTLM auth
`--proxy-header "H: V"`	Proxy-only header
`--noproxy HOSTS`	Comma-separated list of hosts to skip proxy
`--socks4 HOST:PORT`	SOCKS4 proxy
`--socks4a HOST:PORT`	SOCKS4a proxy (resolve DNS remotely)
`--socks5 HOST:PORT`	SOCKS5 proxy
`--socks5-hostname HOST:PORT`	SOCKS5 proxy (resolve DNS remotely)

Miscellaneous

Flag	Description
`-C OFFSET` / `--continue-at`	Resume transfer at offset (`-` for auto)
`-T FILE` / `--upload-file`	Upload file
`-K FILE` / `--config`	Read config from file
`-q`	Do not read `.curlrc`
`--compressed`	Request compressed response (auto decompress)
`--create-dirs`	Create parent directories for output file
`--crlf`	Convert LF to CRLF
`-Z` / `--parallel`	Perform transfers in parallel
`--parallel-max NUM`	Maximum parallel transfers
`--url URL`	Specify URL (alternative to positional)
`-:` / `--next`	Reset state for next URL
`--path-as-is`	Don't squash `..` in URL path
`--raw`	Disable HTTP decoding
`--no-alpn`	Disable ALPN TLS extension
`--no-npn`	Disable NPN TLS extension
`--no-sessionid`	Disable SSL session ID reuse
`--no-buffer`	Disable output buffering
`--stderr FILE`	Redirect stderr to file
`--fail-early`	Fail on first error in batch
`-f` / `--fail`	Fail silently on HTTP errors (no output on 4xx/5xx)
`--fail-with-body`	Fail on HTTP errors but still output body
`-N` / `--no-buffer`	Disable buffering
`--globoff`	Disable URL globbing (brackets)
`--remote-time`	Set local file time to remote file time
`--xattr`	Store metadata in extended file attributes

`--write-out` Format Variables

```
curl -w "HTTP %{http_code}, Time: %{time_total}s, Size: %{size_download} bytes\n" -so /dev/null URL
```

Variable	Description
`%{http_code}`	HTTP response code
`%{http_version}`	HTTP version used
`%{url_effective}`	Last effective URL (after redirects)
`%{redirect_url}`	URL of redirect
`%{num_redirects}`	Number of redirects
`%{time_total}`	Total transfer time (seconds)
`%{time_namelookup}`	DNS resolution time
`%{time_connect}`	TCP connect time
`%{time_appconnect}`	TLS handshake time
`%{time_pretransfer}`	Time until transfer started
`%{time_starttransfer}`	Time to first byte (TTFB)
`%{time_redirect}`	Time spent in redirects
`%{size_download}`	Bytes downloaded
`%{size_upload}`	Bytes uploaded
`%{size_header}`	Header size in bytes
`%{size_request}`	Request size in bytes
`%{speed_download}`	Download speed (bytes/sec)
`%{speed_upload}`	Upload speed (bytes/sec)
`%{content_type}`	Content-Type header
`%{filename_effective}`	Filename used for output
`%{local_ip}`	Local IP used
`%{local_port}`	Local port used
`%{remote_ip}`	Remote IP connected to
`%{remote_port}`	Remote port connected to
`%{ssl_verify_result}`	SSL verification result
`%{scheme}`	URL scheme used
`%{method}`	HTTP method used

Comprehensive Examples

```
# Simple GET request
curl https://example.com

# GET with headers shown
curl -i https://example.com

# HEAD request (headers only)
curl -I https://example.com

# Verbose output (debug)
curl -v https://example.com

# POST with form data
curl -X POST -d "user=alice&pass=secret" https://example.com/login

# POST with JSON
curl -X POST \
  -H "Content-Type: application/json" \
  -d '{"name":"Alice","age":30}' \
  https://api.example.com/users

# POST with JSON from file
curl -X POST \
  -H "Content-Type: application/json" \
  -d @data.json \
  https://api.example.com/users

# PUT request
curl -X PUT \
  -H "Content-Type: application/json" \
  -d '{"name":"Bob"}' \
  https://api.example.com/users/1

# DELETE request
curl -X DELETE https://api.example.com/users/1

# PATCH request
curl -X PATCH \
  -H "Content-Type: application/json" \
  -d '{"name":"Charlie"}' \
  https://api.example.com/users/1

# Upload a file with multipart form
curl -F "file=@photo.jpg" -F "description=My Photo" https://example.com/upload

# Download a file with original name
curl -O https://example.com/file.tar.gz

# Download with a custom filename
curl -o myfile.tar.gz https://example.com/file.tar.gz

# Resume a partial download
curl -C - -O https://example.com/largefile.iso

# Follow redirects
curl -L https://example.com/redirect

# Download with progress bar
curl -# -O https://example.com/file.tar.gz

# Silent download (no output except errors)
curl -sS -O https://example.com/file.tar.gz

# Basic authentication
curl -u admin:password https://api.example.com/data

# Bearer token authentication
```

```
curl -H "Authorization: Bearer YOUR_TOKEN" https://api.example.com/data

# Set custom User-Agent
curl -A "MyApp/1.0" https://example.com

# Set cookies
curl -b "session=abc123" https://example.com

# Save and send cookies across requests
curl -c cookies.txt https://example.com/login -d "user=alice&pass=secret"
curl -b cookies.txt https://example.com/dashboard

# Use a proxy
curl -x http://proxy.example.com:8080 https://example.com
curl -x socks5://proxy.example.com:1080 https://example.com

# Skip SSL verification (for self-signed certs)
curl -k https://self-signed.example.com

# Client certificate authentication
curl --cert client.pem --key client.key https://secure.example.com

# Limit download speed
curl --limit-rate 500K -O https://example.com/file.tar.gz

# Set connection timeout
curl --connect-timeout 5 --max-time 30 https://example.com

# Custom DNS resolution
curl --resolve example.com:443:1.2.3.4 https://example.com

# HTTP/2 (enabled by default when available)
curl --http2 https://example.com

# HTTP/3 (if compiled with support)
curl --http3 https://example.com

# Timing breakdown
curl -o /dev/null -s -w "\
  DNS:        %{time_namelookup}s\n\
  Connect:    %{time_connect}s\n\
  TLS:        %{time_appconnect}s\n\
  TTFB:       %{time_starttransfer}s\n\
  Total:      %{time_total}s\n\
  Size:       %{size_download} bytes\n\
  Speed:      %{speed_download} bytes/sec\n\
  HTTP Code:  %{http_code}\n" https://example.com

# Parallel downloads
curl -Z -O https://example.com/file1.tar.gz -O https://example.com/file2.tar.gz

# Send from stdin
echo "Hello World" | curl -X POST -d @- https://example.com/api

# Request compressed response
curl --compressed https://example.com

# Retry on failure
curl --retry 3 --retry-delay 5 https://example.com

# FTP download
curl -u user:pass ftp://ftp.example.com/file.txt

# FTP upload
curl -u user:pass -T localfile.txt ftp://ftp.example.com/

# SCP download
```

```
curl -u user: --key ~/.ssh/id_rsa scp://example.com/~/file.txt

# SFTP download
curl -u user: --key ~/.ssh/id_rsa sftp://example.com/home/user/file.txt
```

9. `wget` — Non-Interactive Network Downloader

`wget` specializes in downloading files. Supports HTTP, HTTPS, FTP. Key strengths: recursive downloads, resuming, and background operation.

Syntax

```
wget [OPTIONS] URL [URL...]
```

All Major Flags and Options

Download Control

Flag	Description
`-O FILE`	Save to FILE (use `-` for stdout)
`-o LOGFILE`	Log output to LOGFILE
`-a LOGFILE`	Append to LOGFILE
`-c` / `--continue`	Resume partial download
`-N` / `--timestamping`	Only download if remote file is newer
`-T SECONDS` / `--timeout`	Set all timeout values
`--connect-timeout SECONDS`	Connection timeout
`--read-timeout SECONDS`	Read timeout
`--dns-timeout SECONDS`	DNS timeout
`-t NUM` / `--tries`	Number of retries (0 = infinite)
`--retry-connrefused`	Retry on connection refused
`--retry-on-http-error CODES`	Retry on specific HTTP codes
`-w SECONDS` / `--wait`	Wait between retrievals
`--waitretry SECONDS`	Wait between retries
`--random-wait`	Randomize wait time (0.5x to 1.5x of `-w`)
`-Q SIZE` / `--quota`	Download quota (e.g., `100m`)
`--limit-rate RATE`	Limit download speed (e.g., `200k`)
`--bind-address ADDR`	Bind to local address
`-b` / `--background`	Go to background after startup
`--progress TYPE`	Progress type: `bar`, `dot`, `dot:mega`, `dot:giga`
`--show-progress`	Always show progress bar
`--no-verbose`	Turn off verbose without being silent
`-q` / `--quiet`	Quiet mode
`-v` / `--verbose`	Verbose mode
`-d` / `--debug`	Debug mode
`-nv`	Non-verbose (only errors and basic info)
`-i FILE` / `--input-file`	Read URLs from FILE
`-B URL` / `--base`	Base URL for relative links

HTTP Options

Flag	Description
`--header "H: V"`	Add custom header
`--user-agent STRING` / `-U`	Set User-Agent
`--referer URL`	Set Referer header
`--http-user USER`	HTTP username
`--http-password PASS`	HTTP password
`--post-data STRING`	Send POST data
`--post-file FILE`	POST data from file
`--method METHOD`	HTTP method to use
`--body-data STRING`	Request body
`--body-file FILE`	Request body from file
`--no-cookies`	Disable cookies
`--load-cookies FILE`	Load cookies from file
`--save-cookies FILE`	Save cookies to file
`--keep-session-cookies`	Save session cookies too
`--content-disposition`	Honor Content-Disposition filename
`--content-on-error`	Save content on HTTP errors
`--no-http-keep-alive`	Disable HTTP keepalive
`--no-cache`	Don't use cached versions
`--auth-no-challenge`	Send auth without 401 challenge
`--compression auto`	Request compression
`--max-redirect NUM`	Maximum redirects (default: 20; 0 to disable)

SSL/TLS

Flag	Description
`--no-check-certificate`	Skip SSL verification
`--certificate FILE`	Client certificate
`--certificate-type TYPE`	Certificate type
`--private-key FILE`	Private key
`--private-key-type TYPE`	Key type
`--ca-certificate FILE`	CA bundle
`--ca-directory DIR`	CA directory
`--crl-file FILE`	CRL file
`--secure-protocol PROTO`	TLS version: `auto`, `SSLv2`, `SSLv3`, `TLSv1`, `TLSv1_1`, `TLSv1_2`, `TLSv1_3`, `PFS`
`--https-only`	Only follow HTTPS links
`--no-hsts`	Disable HSTS

Recursive Download

Flag	Description
`-r` / `--recursive`	Enable recursive download
`-l DEPTH` / `--level`	Maximum recursion depth (default: 5; 0 = infinite)
`-k` / `--convert-links`	Convert links for local viewing
`-p` / `--page-requisites`	Download all resources needed to display page (CSS, JS, images)
`-m` / `--mirror`	Shorthand for `-r -N -l inf --no-remove-listing`
`-E` / `--adjust-extension`	Add `.html` extension to HTML files
`-H` / `--span-hosts`	Allow spanning across hosts
`-D DOMAINS` / `--domains`	Comma-separated list of allowed domains
`--exclude-domains DOMAINS`	Domains to exclude
`-np` / `--no-parent`	Don't ascend to parent directory
`-A PATTERN` / `--accept`	Accept only files matching pattern (e.g., `*.pdf`)
`-R PATTERN` / `--reject`	Reject files matching pattern
`--accept-regex REGEX`	Accept URLs matching regex
`--reject-regex REGEX`	Reject URLs matching regex
`-I LIST` / `--include-directories`	Include only these directories
`-X LIST` / `--exclude-directories`	Exclude these directories
`--follow-ftp`	Follow FTP links from HTML
`--follow-tags LIST`	HTML tags to follow (e.g., `a,area`)
`--ignore-tags LIST`	HTML tags to ignore
`--ignore-case`	Case-insensitive pattern matching
`--no-host-directories`	Don't create host directories
`--cut-dirs NUM`	Skip NUM directory components
`-nH`	Same as `--no-host-directories`
`--protocol-directories`	Use protocol name in directories
`-nd` / `--no-directories`	Don't create any directories
`-x` / `--force-directories`	Force directory creation
`--robots on/off`	Respect robots.txt (default: on)
`--no-clobber`	Don't overwrite existing files
`--backups NUM`	Keep NUM backups of files

FTP Options

Flag	Description
`--ftp-user USER`	FTP username
`--ftp-password PASS`	FTP password
`--no-passive-ftp`	Use active FTP mode
`--no-remove-listing`	Keep `.listing` files
`--retr-symlinks`	Retrieve symlinks as files
`--no-glob`	Disable FTP globbing
`--preserve-permissions`	Preserve permissions

Proxy

Flag	Description
`-e http_proxy=URL`	Set HTTP proxy
`-e https_proxy=URL`	Set HTTPS proxy
`-e ftp_proxy=URL`	Set FTP proxy
`-e no_proxy=LIST`	No proxy list
`--no-proxy`	Don't use any proxy
`--proxy-user USER:PASS`	Proxy authentication

Examples

```
# Download a single file
wget https://example.com/file.tar.gz

# Download with a custom filename
wget -O custom_name.tar.gz https://example.com/file.tar.gz

# Resume interrupted download
wget -c https://example.com/largefile.iso

# Download in background
wget -b https://example.com/largefile.iso
# Check progress: tail -f wget-log

# Download multiple files from a list
wget -i urls.txt

# Mirror an entire website
wget -m -k -p -E -np https://example.com/docs/

# Download only PDFs from a site
wget -r -l 2 -A "*.pdf" https://example.com/docs/

# Limit download speed
wget --limit-rate=200k https://example.com/file.tar.gz

# Download with retry
wget -t 5 --waitretry=30 https://example.com/file.tar.gz

# Download with authentication
wget --http-user=admin --http-password=secret https://example.com/protected/file.txt

# Download through a proxy
wget -e http_proxy=http://proxy:8080 https://example.com/file.tar.gz

# Skip SSL verification
wget --no-check-certificate https://self-signed.example.com/file.txt

# Download only if newer than local file
wget -N https://example.com/file.tar.gz

# Quiet download (only errors)
wget -q https://example.com/file.tar.gz

# Download with custom User-Agent
wget -U "MyApp/1.0" https://example.com

# Download with custom headers
wget --header="Authorization: Bearer TOKEN" https://api.example.com/data

# Spider mode (check links without downloading)
wget --spider https://example.com

# Download only pages under a specific directory, no parent traversal
wget -r -np -l 3 https://example.com/docs/tutorials/

# Save cookies and reuse them
wget --save-cookies cookies.txt --post-data "user=alice&pass=secret" https://example.com/login
wget --load-cookies cookies.txt https://example.com/dashboard

# FTP download
wget ftp://ftp.example.com/pub/file.tar.gz

# FTP with credentials
wget --ftp-user=user --ftp-password=pass ftp://ftp.example.com/file.txt

# Convert links for offline browsing
wget -r -k -p -l 2 https://example.com
```

10. `dig` — DNS Lookup Utility

`dig` (Domain Information Groper) is the most powerful DNS query tool. It queries DNS servers and returns detailed records.

Syntax

```
dig [@SERVER] NAME [TYPE] [CLASS] [OPTIONS]
```

All Flags and Options

Flag	Description
`@SERVER`	DNS server to query (IP or hostname)
`-b ADDRESS`	Source IP address to bind to
`-p PORT`	Query port (default: 53)
`-q NAME`	Query name (can also be positional)
`-t TYPE`	Query type (can also be positional)
`-c CLASS`	Query class: `IN` (Internet), `CH` (Chaos), `HS` (Hesiod), `ANY`
`-f FILE`	Batch mode — read queries from file
`-k KEYFILE`	TSIG key file for signed queries
`-y [hmac:]name:key`	TSIG key directly
`-x ADDRESS`	Reverse DNS lookup (PTR)
`-4`	Force IPv4
`-6`	Force IPv6
`-m`	Enable memory usage debugging
`-r`	Do not read `~/.digrc`
`-v`	Print version

Query Options (Prefixed with `+` to enable, `+no` to disable)

Option	Description
`+short`	Terse output (just the answer)
`+noall +answer`	Show only the answer section
`+verbose` / `+multiline`	Verbose multi-line output (SOA in human-readable format)
`+trace`	Trace delegation path from root servers
`+nssearch`	Find authoritative nameservers and show SOA from each
`+search`	Use search list from resolv.conf
`+nosearch`	Don't use search list
`+recurse` / `+norecurse`	Enable/disable recursive query (RD bit)
`+tcp`	Use TCP instead of UDP
`+notcp`	Use UDP (default)
`+dnssec`	Request DNSSEC records (DO bit)
`+nodnssec`	Don't request DNSSEC
`+cd`	Set CD (Checking Disabled) flag
`+nocd`	Clear CD flag
`+aaflag` / `+noaaflag`	Set/clear AA (Authoritative Answer) flag
`+adflag` / `+noadflag`	Set/clear AD (Authentic Data) flag
`+all`	Show all sections and flags
`+noall`	Clear all display flags
`+answer` / `+noanswer`	Show/hide answer section
`+authority` / `+noauthority`	Show/hide authority section
`+additional` / `+noadditional`	Show/hide additional section
`+question` / `+noquestion`	Show/hide question section
`+comments` / `+nocomments`	Show/hide comment lines
`+stats` / `+nostats`	Show/hide statistics section
`+qr`	Show outgoing query
`+cmd` / `+nocmd`	Show/hide initial comment showing dig version
`+time=SECONDS`	Query timeout per try
`+tries=NUM`	Number of query attempts
`+retry=NUM`	Number of retries (tries = retry + 1)
`+bufsize=BYTES`	UDP message buffer size (EDNS0)
`+edns=VERSION`	Set EDNS version
`+noedns`	Disable EDNS
`+subnet ADDR/PREFIX`	Set EDNS Client Subnet (ECS)
`+cookie`	Send DNS cookie
`+nocookie`	Don't send cookie
`+nsid`	Request NSID (Name Server IDentifier)
`+identify`	Show IP of server that answered (with `+short`)
`+split=BYTES`	Split hex/base64 output (default: 56)

Option	Description
`+keepopen`	Keep TCP connection open between queries
`+ttlid` / `+nottlid`	Show/hide TTL
`+ttlunits`	Show TTL in human-readable units
`+class` / `+noclass`	Show/hide class
`+rrcomments` / `+norrcomments`	Show/hide per-record comments
`+onesoa`	Show only one SOA in AXFR
`+zflag`	Set Z flag
`+ignore`	Ignore truncation (don't retry with TCP)
`+fail`	Return SERVFAIL on timeout
`+besteffort`	Show partial results on timeout
`+domain=NAME`	Set search domain
`+mapped`	Allow IPv4-mapped IPv6 addresses

DNS Record Types

Type	Description
`A`	IPv4 address
`AAAA`	IPv6 address
`CNAME`	Canonical name (alias)
`MX`	Mail exchange
`NS`	Name server
`PTR`	Pointer (reverse DNS)
`SOA`	Start of Authority
`TXT`	Text record
`SRV`	Service locator
`CAA`	Certification Authority Authorization
`NAPTR`	Naming Authority Pointer
`DNSKEY`	DNSSEC public key
`DS`	Delegation Signer (DNSSEC)
`RRSIG`	Resource Record Signature (DNSSEC)
`NSEC` / `NSEC3`	Next Secure (DNSSEC)
`TLSA`	TLS Authentication (DANE)
`SSHFP`	SSH Fingerprint
`LOC`	Location
`HINFO`	Host information
`SPF`	Sender Policy Framework (deprecated, use TXT)
`AXFR`	Full zone transfer
`IXFR`	Incremental zone transfer
`ANY`	All record types

Examples

```
# Basic A record lookup
dig example.com
# The output contains: QUESTION, ANSWER, AUTHORITY, ADDITIONAL sections

# Just the IP address
dig +short example.com

# Query specific record types
dig example.com A
dig example.com AAAA
dig example.com MX
dig example.com NS
dig example.com TXT
dig example.com SOA
dig example.com CNAME
dig example.com SRV
dig example.com CAA

# Query a specific DNS server
dig @8.8.8.8 example.com
dig @1.1.1.1 example.com
dig @9.9.9.9 example.com

# Reverse DNS lookup
dig -x 8.8.8.8
# Returns: 8.8.8.8.in-addr.arpa.  PTR  dns.google.

# Trace the full delegation chain from root
dig +trace example.com

# Only show the answer section
dig +noall +answer example.com

# Show answer with TTL in human-readable units
dig +noall +answer +ttlunits example.com

# Multi-line verbose SOA
dig +multiline example.com SOA

# Test DNSSEC
dig +dnssec example.com

# Check if domain is signed with DNSSEC
dig +short example.com DNSKEY

# Find all authoritative nameservers
dig +nssearch example.com

# Zone transfer (if permitted)
dig @ns1.example.com example.com AXFR

# Query with EDNS Client Subnet
dig +subnet=1.2.3.0/24 example.com @8.8.8.8

# TCP mode (for large responses)
dig +tcp example.com ANY

# Batch queries from file
# queries.txt contains one query per line:
#   example.com A
#   example.org MX
dig -f queries.txt

# Show only specific sections
dig +noall +answer +authority example.com NS

# Request NSID
```

```
dig +nsid @8.8.8.8 example.com

# Set timeout and retries
dig +time=2 +tries=3 example.com

# Query SRV records (e.g., for SIP, XMPP, LDAP)
dig _sip._tcp.example.com SRV
dig _xmpp-server._tcp.example.com SRV
dig _ldap._tcp.dc._msdcs.example.com SRV

# Check SPF record
dig +short example.com TXT | grep spf

# Check DMARC record
dig +short _dmarc.example.com TXT

# Check DKIM record
dig +short selector._domainkey.example.com TXT

# Show outgoing query
dig +qr example.com

# Compare results from multiple servers
dig @8.8.8.8 +short example.com
dig @1.1.1.1 +short example.com
dig @9.9.9.9 +short example.com
```

11. `nslookup` — DNS Query Tool

`nslookup` is an older, simpler DNS query tool. Less detailed than `dig` but still commonly used.

Syntax

```
nslookup [OPTIONS] [NAME] [SERVER]
```

Options

Flag	Description
`-type=TYPE`	Query type (A, AAAA, MX, NS, SOA, TXT, SRV, PTR, ANY, etc.)
`-debug`	Show debugging info
`-port=PORT`	Query port
`-timeout=SECONDS`	Query timeout
`-retry=NUM`	Number of retries
`-domain=NAME`	Default domain
`-search`	Use search list
`-class=CLASS`	Query class

Interactive Mode Commands

```
nslookup                          # Enter interactive mode
> server 8.8.8.8                  # Change DNS server
> set type=MX                     # Set query type
> example.com                     # Perform query
> set debug                       # Enable debug output
> set nodebug                     # Disable debug output
> set all                         # Show current settings
> exit                            # Exit
```

Examples

```
# Basic lookup
nslookup example.com

# Use a specific DNS server
nslookup example.com 8.8.8.8

# MX record lookup
nslookup -type=MX example.com

# NS record lookup
nslookup -type=NS example.com

# TXT record lookup
nslookup -type=TXT example.com

# SOA record lookup
nslookup -type=SOA example.com

# Reverse DNS
nslookup 8.8.8.8

# AAAA (IPv6) lookup
nslookup -type=AAAA example.com

# With debug info
nslookup -debug example.com

# SRV record
nslookup -type=SRV _sip._tcp.example.com
```

12. `host` — Simple DNS Lookup

`host` provides clean, human-readable DNS output.

Syntax

```
host [OPTIONS] NAME [SERVER]
```

All Flags

Flag	Description
`-t TYPE`	Query type
`-a`	Same as `-t ANY`
`-d`	Verbose (same as `-v`)
`-l ZONE`	Zone transfer
`-r`	Non-recursive query
`-T`	Use TCP
`-4`	Force IPv4
`-6`	Force IPv6
`-v`	Verbose output (similar to dig)
`-w`	Wait forever for response
`-W SECONDS`	Query timeout
`-R NUM`	Number of retries
`-m FLAG`	Memory debugging: `record`, `usage`, `trace`
`-c CLASS`	Query class
`-C`	Check SOA consistency on all authoritative nameservers
`-N NUM`	Number of dots that must appear to be considered absolute name
`-s`	Do NOT send query to next nameserver on SERVFAIL

Examples

```
# Basic lookup
host example.com
# Output:
# example.com has address 93.184.216.34
# example.com has IPv6 address 2606:2800:220:1:248:1893:25c8:1946
# example.com mail is handled by 0 .

# Reverse lookup
host 8.8.8.8
# Output: 8.8.8.8.in-addr.arpa domain name pointer dns.google.

# MX records
host -t MX example.com

# NS records
host -t NS example.com

# TXT records
host -t TXT example.com

# SOA record
host -t SOA example.com

# Use a specific DNS server
host example.com 8.8.8.8

# Verbose output (dig-like)
host -v example.com

# All record types
host -a example.com

# SOA consistency check
host -C example.com

# Zone transfer
host -l example.com ns1.example.com

# TCP mode
host -T example.com
```

13. `nmap` — Network Scanner

`nmap` (Network Mapper) is the industry-standard tool for network discovery and security auditing.

Syntax

```
nmap [SCAN TYPE] [OPTIONS] TARGET
```

Target Specification

```
nmap 192.168.1.1                    # Single host
nmap 192.168.1.1 192.168.1.2       # Multiple hosts
nmap 192.168.1.0/24                 # CIDR subnet
nmap 192.168.1.1-254               # IP range
nmap 192.168.1.*                    # Wildcard
nmap 192.168.1,2.0-255             # Octet ranges
nmap -iL targets.txt               # Read from file
nmap --exclude 192.168.1.1         # Exclude hosts
nmap --excludefile exclude.txt     # Exclude from file
nmap 192.168.1.0/24 --exclude 192.168.1.1
```

Host Discovery

Flag	Description
`-sn`	Ping scan (no port scan) — host discovery only
`-Pn`	Skip host discovery (treat all as online)
`-PS PORTLIST`	TCP SYN ping on given ports
`-PA PORTLIST`	TCP ACK ping on given ports
`-PU PORTLIST`	UDP ping on given ports
`-PY PORTLIST`	SCTP INIT ping
`-PE`	ICMP Echo ping
`-PP`	ICMP Timestamp ping
`-PM`	ICMP Address Mask ping
`-PO PROTOCOLS`	IP Protocol ping
`-PR`	ARP ping (local network)
`--disable-arp-ping`	Disable ARP ping
`-n`	Never do DNS resolution
`-R`	Always resolve DNS
`--dns-servers SERVERS`	Custom DNS servers
`--system-dns`	Use OS DNS resolver
`--traceroute`	Trace hop path

Scan Techniques

Flag	Description
`-sS`	TCP SYN scan (default, stealthy, root required)
`-sT`	TCP Connect scan (full 3-way handshake, no root needed)
`-sU`	UDP scan
`-sA`	TCP ACK scan (detect filtered ports/firewalls)
`-sW`	TCP Window scan (like ACK but detects open ports on some systems)
`-sM`	TCP Maimon scan (FIN/ACK)
`-sN`	TCP Null scan (no flags set)
`-sF`	TCP FIN scan
`-sX`	TCP Xmas scan (FIN+PSH+URG)
`-sI ZOMBIE`	Idle scan (extremely stealthy, uses zombie host)
`-sY`	SCTP INIT scan
`-sZ`	SCTP COOKIE-ECHO scan
`-sO`	IP Protocol scan (detect supported IP protocols)
`-b FTP_HOST`	FTP bounce scan
`--scanflags FLAGS`	Custom TCP flags

Port Specification

Flag	Description
`-p PORTS`	Scan these ports. E.g.: `-p 22`, `-p 1-65535`, `-p 80,443`, `-p U:53,T:22`
`-p-`	Scan all 65535 ports
`--top-ports N`	Scan N most common ports
`-F`	Fast scan (100 most common ports)
`-r`	Scan ports sequentially (don't randomize)
`--port-ratio RATIO`	Scan ports with ratio ≥ RATIO

Service/Version Detection

Flag	Description
`-sV`	Probe open ports for service/version info
`--version-intensity LEVEL`	0 (light) to 9 (try all probes)
`--version-light`	Same as `--version-intensity 2`
`--version-all`	Same as `--version-intensity 9`
`--version-trace`	Show detailed version scan activity

OS Detection

Flag	Description
`-O`	Enable OS detection
`--osscan-limit`	Only try OS detection on promising hosts
`--osscan-guess`	Guess OS more aggressively
`--max-os-tries N`	Maximum OS detection tries

Timing and Performance

Flag	Description
`-T0`	Paranoid (very slow, IDS evasion)
`-T1`	Sneaky (slow, IDS evasion)
`-T2`	Polite (slowed down)
`-T3`	Normal (default)
`-T4`	Aggressive (faster)
`-T5`	Insane (fastest, may miss ports)
`--min-hostgroup N`	Minimum hosts to scan in parallel
`--max-hostgroup N`	Maximum hosts to scan in parallel
`--min-parallelism N`	Minimum probe parallelism
`--max-parallelism N`	Maximum probe parallelism
`--min-rtt-timeout MS`	Minimum probe RTT timeout
`--max-rtt-timeout MS`	Maximum probe RTT timeout
`--initial-rtt-timeout MS`	Initial probe RTT timeout
`--max-retries N`	Maximum probe retransmissions
`--host-timeout MS`	Give up on host after this time
`--scan-delay MS`	Adjust delay between probes
`--max-scan-delay MS`	Maximum delay between probes
`--min-rate N`	Minimum packets per second
`--max-rate N`	Maximum packets per second

Firewall/IDS Evasion

Flag	Description
`-f`	Fragment packets
`--mtu VALUE`	Set custom MTU
`-D DECOYS`	Cloak scan with decoys (e.g., `-D RND:5`)
`-S SOURCE`	Spoof source address
`-e INTERFACE`	Use specified interface
`--source-port PORT`	Use specified source port
`--proxies URL`	Relay connections through proxies
`--data HEX`	Append custom hex data
`--data-string STRING`	Append custom ASCII data
`--data-length NUM`	Append random data of given length
`--ip-options OPTIONS`	Set IP options
`--ttl VALUE`	Set IP TTL
`--spoof-mac MAC`	Spoof MAC address
`--badsum`	Send with bad checksums
`--adler32`	Use Adler32 for SCTP checksums

Scripts (NSE — Nmap Scripting Engine)

Flag	Description
`-sC`	Run default scripts (same as `--script=default`)
`--script SCRIPTS`	Run specified scripts. Categories: `auth`, `broadcast`, `brute`, `default`, `discovery`, `dos`, `exploit`, `external`, `fuzzer`, `intrusive`, `malware`, `safe`, `version`, `vuln`
`--script-args ARGS`	Script arguments (e.g., `user=admin,pass=secret`)
`--script-args-file FILE`	Load script args from file
`--script-trace`	Show all sent/received script data
`--script-updatedb`	Update the script database
`--script-help SCRIPTS`	Show help for scripts

Output

Flag	Description
`-oN FILE`	Normal output
`-oX FILE`	XML output
`-oG FILE`	Grepable output
`-oS FILE`	ScRiPt KiDDi3 output
`-oA BASENAME`	Output in all major formats
`-v`	Increase verbosity (use `-vv` for more)
`-d`	Increase debugging (use `-dd` for more)
`--reason`	Display reason for port state
`--open`	Show only open ports
`--packet-trace`	Show all packets sent/received
`--iflist`	Print host interfaces and routes
`--append-output`	Append to output files
`--resume FILE`	Resume aborted scan
`--stylesheet FILE`	XSL stylesheet for XML output
`--webxml`	Use Nmap.org stylesheet
`--no-stylesheet`	No XSL stylesheet
`--stats-every TIME`	Print stats periodically

Miscellaneous

Flag	Description
`-6`	Enable IPv6 scanning
`-A`	Aggressive scan: OS detection + version + scripts + traceroute
`-V`	Print version number
`-h`	Help
`--datadir DIR`	Nmap data directory
`--send-eth`	Send using raw Ethernet frames
`--send-ip`	Send using raw IP packets
`--privileged`	Assume user is privileged
`--unprivileged`	Assume user is not privileged
`--release-memory`	Release memory before quitting

Comprehensive Examples

```
# Quick scan of common ports
nmap 192.168.1.1

# Scan all 65535 ports
nmap -p- 192.168.1.1

# Scan specific ports
nmap -p 22,80,443,3306,5432,8080 192.168.1.1

# Scan port range
nmap -p 1-1000 192.168.1.1

# Fast scan (100 most common ports)
nmap -F 192.168.1.0/24

# Ping sweep (discover live hosts only)
nmap -sn 192.168.1.0/24

# SYN scan with version detection
sudo nmap -sS -sV 192.168.1.1

# Full aggressive scan
sudo nmap -A 192.168.1.1

# OS detection
sudo nmap -O 192.168.1.1

# UDP scan
sudo nmap -sU -p 53,67,68,69,123,161,162,514 192.168.1.1

# Combined TCP + UDP scan
sudo nmap -sS -sU -p T:80,443,U:53,161 192.168.1.1

# Run default NSE scripts
nmap -sC 192.168.1.1

# Scan + version + default scripts
nmap -sV -sC 192.168.1.1

# Run vulnerability scripts
nmap --script vuln 192.168.1.1

# Run specific scripts
nmap --script http-title,http-headers 192.168.1.1
nmap --script "http-*" 192.168.1.1
nmap --script "not intrusive" 192.168.1.1
nmap --script "safe and discovery" 192.168.1.1

# Brute force SSH
nmap --script ssh-brute -p 22 192.168.1.1

# Detect SSL vulnerabilities
nmap --script ssl-heartbleed,ssl-poodle -p 443 192.168.1.1

# Enumerate HTTP
nmap --script http-enum -p 80 192.168.1.1

# Service banner grabbing
nmap -sV --version-intensity 5 192.168.1.1

# Stealth scan with decoys
sudo nmap -sS -D RND:5 192.168.1.1

# Scan using a specific source port (bypass some firewalls)
sudo nmap --source-port 53 192.168.1.1

# Fragment packets (evade IDS)
```

```
sudo nmap -f 192.168.1.1

# Timing: aggressive
nmap -T4 192.168.1.0/24

# Timing: paranoid (IDS evasion)
nmap -T0 192.168.1.1

# Output to all formats
nmap -oA scan_results 192.168.1.0/24

# XML output
nmap -oX scan.xml 192.168.1.0/24

# Show only open ports
nmap --open 192.168.1.0/24

# Show reason for port state
nmap --reason 192.168.1.1

# Idle scan (extremely stealthy)
sudo nmap -sI zombie.example.com 192.168.1.1

# Scan IPv6
nmap -6 2001:db8::1

# Scan top 20 ports on entire subnet, fast
nmap --top-ports 20 -T4 10.0.0.0/24

# List network interfaces
nmap --iflist

# Rate-limited scan
nmap --max-rate 100 192.168.1.0/24

# Resume an interrupted scan
nmap --resume scan_results.gnmap
```

14. `tcpdump` — Packet Capture & Analysis

`tcpdump` captures packets from a network interface. It is the standard command-line packet analyzer.

Syntax

```
tcpdump [OPTIONS] [EXPRESSION]
```

All Flags and Options

Flag	Description
`-i INTERFACE`	Capture on specific interface. Use `any` for all interfaces
`-c COUNT`	Capture COUNT packets then stop
`-w FILE`	Write raw packets to file (pcap format)
`-r FILE`	Read packets from file
`-n`	Don't resolve hostnames
`-nn`	Don't resolve hostnames OR port names
`-v`	Verbose output
`-vv`	More verbose
`-vvv`	Maximum verbosity
`-q`	Quiet (less protocol info)
`-e`	Print link-layer (MAC) header
`-A`	Print packet payload in ASCII
`-X`	Print packet payload in hex and ASCII
`-XX`	Print packet with link-level header in hex and ASCII
`-D`	List available interfaces
`-l`	Line-buffered (useful for piping)
`-t`	Don't print timestamp
`-tt`	Print Unix timestamp
`-ttt`	Print delta between packets
`-tttt`	Print date + time
`-ttttt`	Print delta from first packet
`-s SNAPLEN`	Capture SNAPLEN bytes per packet (default: 262144). Use `-s 0` for full packet
`-S`	Print absolute TCP sequence numbers
`-C FILESIZE`	Rotate capture files at FILESIZE (MB)
`-G SECONDS`	Rotate capture files every SECONDS
`-W COUNT`	Keep only COUNT rotation files
`-Z USER`	Drop privileges to USER
`-K`	Don't verify checksums
`-p`	Don't set promiscuous mode
`-U`	Packet-buffered output (write each packet immediately)
`-B BUFFER`	Set kernel capture buffer size (KB)
`-F FILE`	Read BPF filter from file
`-Q DIRECTION`	Capture direction: `in`, `out`, `inout`
`--direction DIRECTION`	Same as `-Q`
`-j TSTAMP_TYPE`	Timestamp type
`--list-time-stamp-types`	List available timestamp types
`--time-stamp-precision PREC`	`micro` or `nano`
`--immediate-mode`	Deliver packets as captured

Flag	Description
`--print`	Print parsed output even with `-w`
`--version`	Print version
`--count`	Print only a count of packets

BPF Filter Expressions

BPF (Berkeley Packet Filter) expressions define which packets to capture.

Protocol Filters

```
tcpdump tcp                          # TCP only
tcpdump udp                          # UDP only
tcpdump icmp                         # ICMP only
tcpdump arp                          # ARP only
tcpdump ip                           # IPv4 only
tcpdump ip6                          # IPv6 only
tcpdump vlan                         # VLAN tagged
```

Host Filters

```
tcpdump host 192.168.1.100           # Traffic to/from host
tcpdump src host 192.168.1.100       # Traffic FROM host
tcpdump dst host 192.168.1.100       # Traffic TO host
tcpdump net 192.168.1.0/24           # Traffic to/from network
tcpdump src net 10.0.0.0/8           # Traffic FROM network
tcpdump dst net 10.0.0.0/8           # Traffic TO network
```

Port Filters

```
tcpdump port 80                      # Traffic on port 80
tcpdump src port 443                 # Traffic FROM port 443
tcpdump dst port 22                  # Traffic TO port 22
tcpdump portrange 1024-65535         # Port range
tcpdump tcp port 80                  # TCP traffic on port 80
tcpdump udp port 53                  # UDP traffic on port 53
```

Logical Operators

```
tcpdump host 192.168.1.1 and port 80          # AND
tcpdump host 192.168.1.1 or host 192.168.1.2  # OR
tcpdump not port 22                             # NOT
tcpdump '(host 192.168.1.1 or host 192.168.1.2) and port 80'  # Grouping
```

TCP Flag Filters

```
tcpdump 'tcp[tcpflags] & tcp-syn != 0'        # SYN packets
tcpdump 'tcp[tcpflags] & tcp-ack != 0'        # ACK packets
tcpdump 'tcp[tcpflags] & tcp-fin != 0'        # FIN packets
tcpdump 'tcp[tcpflags] & tcp-rst != 0'        # RST packets
tcpdump 'tcp[tcpflags] & tcp-push != 0'       # PSH packets
tcpdump 'tcp[tcpflags] & (tcp-syn|tcp-ack) = (tcp-syn|tcp-ack)'  # SYN-ACK
tcpdump 'tcp[tcpflags] = tcp-syn'             # SYN only (no other flags)
```

Size Filters

```
tcpdump greater 1000                  # Packets larger than 1000 bytes
tcpdump less 64                       # Packets smaller than 64 bytes
tcpdump 'len > 500'                   # Same as greater
```

Advanced Filters

```
# VLAN tagged traffic
tcpdump vlan

# Specific VLAN ID
tcpdump 'vlan 100'

# Broadcast/multicast
tcpdump broadcast
tcpdump multicast
tcpdump ether broadcast

# Specific MAC address
tcpdump ether host 00:11:22:33:44:55
tcpdump ether src 00:11:22:33:44:55
tcpdump ether dst ff:ff:ff:ff:ff:ff

# IP fragment
tcpdump 'ip[6:2] & 0x1fff != 0'

# TTL = 1 (traceroute detection)
tcpdump 'ip[8] = 1'

# HTTP GET requests
tcpdump -A 'tcp port 80 and tcp[((tcp[12:1] & 0xf0) >> 2):4] = 0x47455420'

# DNS queries
tcpdump -n udp port 53
```

Comprehensive Examples

```
# Capture all traffic on eth0
sudo tcpdump -i eth0

# Capture with no DNS/port resolution, verbose
sudo tcpdump -i eth0 -nn -vv

# Capture first 100 packets on any interface
sudo tcpdump -i any -c 100

# Save capture to file
sudo tcpdump -i eth0 -w capture.pcap

# Read from saved file
tcpdump -r capture.pcap

# Capture only HTTP traffic
sudo tcpdump -i eth0 tcp port 80

# Capture HTTP and HTTPS
sudo tcpdump -i eth0 'tcp port 80 or tcp port 443'

# Capture DNS traffic
sudo tcpdump -i eth0 udp port 53

# Capture all traffic to/from a host
sudo tcpdump -i eth0 host 192.168.1.100

# Capture SSH traffic
sudo tcpdump -i eth0 tcp port 22

# Capture with packet content in ASCII
sudo tcpdump -i eth0 -A port 80

# Capture with hex+ASCII dump
sudo tcpdump -i eth0 -X port 80

# Capture with MAC addresses shown
sudo tcpdump -i eth0 -e

# Capture full packets (no truncation)
sudo tcpdump -i eth0 -s 0

# Capture with timestamps and write to file
sudo tcpdump -i eth0 -tttt -w timestamped.pcap

# Capture traffic between two specific hosts
sudo tcpdump -i eth0 'host 192.168.1.1 and host 192.168.1.2'

# Capture non-SSH traffic (exclude SSH)
sudo tcpdump -i eth0 'not port 22'

# Capture SYN packets only (new connections)
sudo tcpdump -i eth0 'tcp[tcpflags] = tcp-syn'

# Capture RST packets (connection resets)
sudo tcpdump -i eth0 'tcp[tcpflags] & tcp-rst != 0'

# Line-buffered output for piping
sudo tcpdump -i eth0 -l port 80 | grep "GET"

# Rotate files every 100MB, keep 10 files
sudo tcpdump -i eth0 -w capture.pcap -C 100 -W 10

# Rotate files every 3600 seconds (1 hour)
sudo tcpdump -i eth0 -w 'capture_%Y%m%d_%H%M%S.pcap' -G 3600

# Capture only incoming traffic
```

```
sudo tcpdump -i eth0 -Q in

# Capture only outgoing traffic
sudo tcpdump -i eth0 -Q out

# Capture ICMP (ping)
sudo tcpdump -i eth0 icmp

# Capture ARP
sudo tcpdump -i eth0 arp

# Capture DHCP
sudo tcpdump -i eth0 udp port 67 or udp port 68

# Count packets matching filter
sudo tcpdump -i eth0 --count tcp port 80

# Print delta between packets
sudo tcpdump -i eth0 -ttt port 443

# List available interfaces
tcpdump -D
```

15. `iptables` — Firewall (Netfilter)

`iptables` manages the Linux kernel's packet filtering framework (Netfilter). It operates on chains of rules organized in tables.

Tables

Table	Purpose
`filter`	Default. Packet filtering (accept/drop/reject)
`nat`	Network Address Translation
`mangle`	Packet alteration (TOS, TTL, mark)
`raw`	Exemptions from connection tracking
`security`	SELinux security marking

Chains

Chain	Table	Description
`INPUT`	filter, nat, mangle, security	Incoming packets destined for the local machine
`FORWARD`	filter, mangle, security	Packets being routed through the machine
`OUTPUT`	filter, nat, mangle, raw, security	Outgoing packets from the local machine
`PREROUTING`	nat, mangle, raw	Before routing decision
`POSTROUTING`	nat, mangle	After routing decision

Syntax

```
iptables [-t TABLE] COMMAND CHAIN [MATCH] [TARGET]
```

Commands

Flag	Description
`-A CHAIN`	Append rule to chain
`-I CHAIN [NUM]`	Insert rule at position (default: 1)
`-D CHAIN RULE`	Delete rule by specification
`-D CHAIN NUM`	Delete rule by number
`-R CHAIN NUM RULE`	Replace rule at position
`-L [CHAIN]`	List rules
`-S [CHAIN]`	Print rules as iptables commands
`-F [CHAIN]`	Flush (delete all rules) in chain
`-Z [CHAIN]`	Zero packet/byte counters
`-N CHAIN`	Create new user-defined chain
`-X [CHAIN]`	Delete empty user-defined chain
`-P CHAIN TARGET`	Set default policy for built-in chain
`-E OLD NEW`	Rename user-defined chain
`-C CHAIN RULE`	Check if rule exists

Match Options

Flag	Description
`-p PROTO`	Protocol: `tcp`, `udp`, `icmp`, `icmpv6`, `esp`, `ah`, `sctp`, `udplite`, `all`
`-s SOURCE`	Source address/network (e.g., `192.168.1.0/24`)
`-d DEST`	Destination address/network
`-i IFACE`	Input interface (for INPUT/FORWARD/PREROUTING)
`-o IFACE`	Output interface (for OUTPUT/FORWARD/POSTROUTING)
`-f`	Match second and further fragments
`!`	Negate the match (e.g., `! -s 10.0.0.0/8`)

TCP Match Extensions (`-p tcp`)

Flag	Description
`--sport PORT[:PORT]`	Source port or range
`--dport PORT[:PORT]`	Destination port or range
`--tcp-flags MASK COMP`	Match TCP flags (e.g., `SYN,ACK,FIN,RST SYN`)
`--syn`	Match SYN packets (new connections)
`--tcp-option NUM`	Match TCP option

UDP Match Extensions (`-p udp`)

Flag	Description
`--sport PORT[:PORT]`	Source port or range
`--dport PORT[:PORT]`	Destination port or range

ICMP Match Extensions (`-p icmp`)

Flag	Description
`--icmp-type TYPE`	ICMP type: `echo-request`, `echo-reply`, `destination-unreachable`, `time-exceeded`, etc.

Match Modules (`-m MODULE`)

Module	Flags	Description
`state`	`--state NEW,ESTABLISHED,RELATED,INVALID`	Connection tracking state
`conntrack`	`--ctstate`, `--ctproto`, `--ctorigsrc`, etc.	Advanced connection tracking
`multiport`	`--sports`, `--dports`, `--ports`	Match multiple ports (up to 15)
`iprange`	`--src-range IP-IP`, `--dst-range IP-IP`	Match IP range
`mac`	`--mac-source MAC`	Match source MAC address
`limit`	`--limit RATE`, `--limit-burst NUM`	Rate limiting
`hashlimit`	`--hashlimit-upto`, `--hashlimit-name`, etc.	Per-address rate limiting
`recent`	`--name`, `--set`, `--rcheck`, `--update`, `--seconds`, `--hit-count`	Track recent connections
`string`	`--string PATTERN`, `--algo {bm,kmp}`	Match packet content
`comment`	`--comment "TEXT"`	Add comment to rule
`mark`	`--mark VALUE[/MASK]`	Match packet mark
`owner`	`--uid-owner`, `--gid-owner`	Match packet owner (OUTPUT only)
`length`	`--length MIN:MAX`	Match packet length
`ttl`	`--ttl-eq`, `--ttl-gt`, `--ttl-lt`	Match TTL
`tos`	`--tos VALUE`	Match TOS field
`dscp`	`--dscp VALUE`, `--dscp-class CLASS`	Match DSCP field
`time`	`--timestart`, `--timestop`, `--days`, `--monthdays`	Time-based matching
`connlimit`	`--connlimit-above N`, `--connlimit-mask BITS`	Limit concurrent connections
`set`	`--match-set NAME FLAGS`	Match against ipset
`geoip`	`--src-cc`, `--dst-cc`	Match by country (xt_geoip)
`addrtype`	`--src-type TYPE`, `--dst-type TYPE`	Match address type (LOCAL, UNICAST, etc.)
`physdev`	`--physdev-in`, `--physdev-out`	Match bridge physical device
`policy`	`--pol`, `--dir`, `--strict`	Match IPsec policy
`u32`	`--u32 TEST`	Match arbitrary bytes

Targets (`-j TARGET`)

Target	Description
`ACCEPT`	Accept the packet
`DROP`	Silently drop the packet
`REJECT`	Reject with ICMP error (configurable with `--reject-with`)
`LOG`	Log the packet (continues processing)
`RETURN`	Return from user chain to calling chain
`QUEUE`	Queue to userspace
`NFQUEUE`	Queue to userspace via nfnetlink
`MASQUERADE`	NAT masquerade (dynamic SNAT, nat/POSTROUTING)
`SNAT`	Source NAT (`--to-source IP[:PORT]`, nat/POSTROUTING)
`DNAT`	Destination NAT (`--to-destination IP[:PORT]`, nat/PREROUTING)
`REDIRECT`	Redirect to local port (`--to-ports PORT`, nat/PREROUTING)
`MARK`	Set packet mark (`--set-mark VALUE`)
`CONNMARK`	Set connection mark
`TOS`	Set TOS field
`DSCP`	Set DSCP field
`TTL`	Modify TTL
`TCPMSS`	Clamp MSS (`--clamp-mss-to-pmtu`)
`NOTRACK`	Skip connection tracking (raw table)
`CLASSIFY`	Set packet priority
`TPROXY`	Transparent proxy
`CT`	Connection tracking helper assignment
`AUDIT`	Audit logging
`SECMARK`	Set security mark

Display Options

Flag	Description
`-L`	List rules
`-n`	Numeric output
`-v`	Verbose (show counters, interface)
`--line-numbers`	Show rule numbers
`-x`	Exact byte/packet counters

Comprehensive Examples

```
# List all rules (verbose, numeric, with line numbers)
sudo iptables -L -n -v --line-numbers

# List rules as commands (easy to save/restore)
sudo iptables -S

# List NAT rules
sudo iptables -t nat -L -n -v

# Set default policies (secure baseline)
sudo iptables -P INPUT DROP
sudo iptables -P FORWARD DROP
sudo iptables -P OUTPUT ACCEPT

# Allow loopback
sudo iptables -A INPUT -i lo -j ACCEPT
sudo iptables -A OUTPUT -o lo -j ACCEPT

# Allow established and related connections
sudo iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT

# Allow SSH
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT

# Allow HTTP and HTTPS
sudo iptables -A INPUT -p tcp -m multiport --dports 80,443 -j ACCEPT

# Allow DNS
sudo iptables -A INPUT -p udp --dport 53 -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 53 -j ACCEPT

# Allow ICMP (ping)
sudo iptables -A INPUT -p icmp --icmp-type echo-request -j ACCEPT

# Block a specific IP
sudo iptables -A INPUT -s 10.0.0.5 -j DROP

# Block a subnet
sudo iptables -A INPUT -s 10.99.0.0/16 -j DROP

# Reject (send ICMP unreachable) instead of silent drop
sudo iptables -A INPUT -s 10.0.0.5 -j REJECT --reject-with icmp-port-unreachable

# Rate limit incoming connections (anti-DDoS)
sudo iptables -A INPUT -p tcp --dport 80 -m limit --limit 25/minute --limit-burst 100 -j ACCEPT

# Limit SSH connections (prevent brute force)
sudo iptables -A INPUT -p tcp --dport 22 -m state --state NEW -m recent --set --name SSH
sudo iptables -A INPUT -p tcp --dport 22 -m state --state NEW -m recent --update --seconds 60 --hitcount
4 --name SSH -j DROP

# Limit concurrent connections from a single IP
sudo iptables -A INPUT -p tcp --dport 80 -m connlimit --connlimit-above 50 -j REJECT

# Log dropped packets
sudo iptables -A INPUT -j LOG --log-prefix "IPTables-Dropped: " --log-level 4
sudo iptables -A INPUT -j DROP

# NAT / Masquerading (enable internet sharing)
sudo iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE
sudo sysctl -w net.ipv4.ip_forward=1

# SNAT (static source NAT)
sudo iptables -t nat -A POSTROUTING -s 192.168.1.0/24 -o eth0 -j SNAT --to-source 203.0.113.1

# DNAT / Port forwarding
sudo iptables -t nat -A PREROUTING -i eth0 -p tcp --dport 8080 -j DNAT --to-destination 192.168.1.100:80
```

```
sudo iptables -A FORWARD -p tcp -d 192.168.1.100 --dport 80 -j ACCEPT

# Redirect port (local port forwarding)
sudo iptables -t nat -A PREROUTING -p tcp --dport 80 -j REDIRECT --to-ports 8080

# Allow traffic from specific MAC
sudo iptables -A INPUT -m mac --mac-source 00:11:22:33:44:55 -j ACCEPT

# Match IP range
sudo iptables -A INPUT -m iprange --src-range 192.168.1.100-192.168.1.200 -j ACCEPT

# Time-based rule (allow HTTP only during business hours)
sudo iptables -A INPUT -p tcp --dport 80 -m time --timestart 08:00 --timestop 18:00 --days Mon,Tue,Wed,Thu
,Fri -j ACCEPT

# Match by owner (OUTPUT chain only)
sudo iptables -A OUTPUT -m owner --uid-owner nobody -j DROP

# Clamp MSS for VPN/tunnel issues
sudo iptables -t mangle -A FORWARD -p tcp --tcp-flags SYN,RST SYN -j TCPMSS --clamp-mss-to-pmtu

# Mark packets for policy routing
sudo iptables -t mangle -A PREROUTING -s 192.168.1.0/24 -j MARK --set-mark 1

# Match string in packet content
sudo iptables -A INPUT -m string --string "X-Malicious" --algo bm -j DROP

# Insert rule at position 1
sudo iptables -I INPUT 1 -p tcp --dport 22 -j ACCEPT

# Delete rule by number
sudo iptables -D INPUT 3

# Delete rule by specification
sudo iptables -D INPUT -p tcp --dport 80 -j ACCEPT

# Flush all rules
sudo iptables -F

# Flush NAT rules
sudo iptables -t nat -F

# Zero counters
sudo iptables -Z

# Create custom chain
sudo iptables -N MY_CHAIN
sudo iptables -A MY_CHAIN -p tcp --dport 80 -j ACCEPT
sudo iptables -A MY_CHAIN -j RETURN
sudo iptables -A INPUT -j MY_CHAIN

# Save rules
sudo iptables-save > /etc/iptables/rules.v4

# Restore rules
sudo iptables-restore < /etc/iptables/rules.v4

# IPv6 rules (use ip6tables)
sudo ip6tables -A INPUT -p tcp --dport 22 -j ACCEPT
sudo ip6tables -L -n -v
```

16. `ufw` — Uncomplicated Firewall

A user-friendly frontend for `iptables`, commonly used on Ubuntu/Debian.

All Commands

```
# Enable/Disable
sudo ufw enable                      # Enable firewall
sudo ufw disable                     # Disable firewall
sudo ufw reset                       # Reset to defaults

# Status
sudo ufw status                      # Show status and rules
sudo ufw status verbose              # Verbose status
sudo ufw status numbered             # Show rules with numbers

# Default policies
sudo ufw default deny incoming       # Block all incoming
sudo ufw default allow outgoing      # Allow all outgoing
sudo ufw default deny forward        # Block forwarding

# Allow/Deny by port
sudo ufw allow 22                    # Allow port 22 (TCP+UDP)
sudo ufw allow 22/tcp                # Allow port 22 TCP only
sudo ufw allow 80,443/tcp            # Allow multiple ports
sudo ufw allow 6000:6007/tcp         # Allow port range
sudo ufw deny 23                     # Deny port 23

# Allow/Deny by service name
sudo ufw allow ssh
sudo ufw allow http
sudo ufw allow https
sudo ufw allow 'Apache Full'

# Allow/Deny by IP
sudo ufw allow from 192.168.1.100
sudo ufw deny from 10.0.0.5
sudo ufw allow from 192.168.1.0/24

# Allow from IP to specific port
sudo ufw allow from 192.168.1.100 to any port 22
sudo ufw allow from 192.168.1.0/24 to any port 80 proto tcp

# Allow to specific interface
sudo ufw allow in on eth0 to any port 80

# Rate limiting (prevent brute force)
sudo ufw limit ssh                   # Limit SSH connections

# Delete rules
sudo ufw delete allow 80
sudo ufw delete 3                    # Delete by number

# Application profiles
sudo ufw app list                    # List available profiles
sudo ufw app info 'OpenSSH'         # Show profile details
sudo ufw allow 'OpenSSH'

# Logging
sudo ufw logging on                  # Enable logging
sudo ufw logging off                 # Disable logging
sudo ufw logging low                 # Log blocked
sudo ufw logging medium              # Log blocked + invalid
sudo ufw logging high                # Log all
sudo ufw logging full                # Log everything

# Routing/Forwarding
sudo ufw route allow in on eth0 out on eth1 to 10.0.0.0/24
```

17. `firewall-cmd` — Firewalld CLI (RHEL/CentOS/Fedora)

Commands

```
# Status
sudo firewall-cmd --state
sudo firewall-cmd --get-active-zones
sudo firewall-cmd --get-zones
sudo firewall-cmd --get-default-zone
sudo firewall-cmd --list-all
sudo firewall-cmd --list-all-zones

# Zone management
sudo firewall-cmd --set-default-zone=public
sudo firewall-cmd --zone=public --list-all
sudo firewall-cmd --zone=trusted --add-interface=eth1

# Services
sudo firewall-cmd --list-services
sudo firewall-cmd --get-services                        # All available services
sudo firewall-cmd --permanent --add-service=http
sudo firewall-cmd --permanent --add-service=https
sudo firewall-cmd --permanent --remove-service=http
sudo firewall-cmd --permanent --zone=public --add-service=ssh

# Ports
sudo firewall-cmd --permanent --add-port=8080/tcp
sudo firewall-cmd --permanent --add-port=5000-5100/tcp
sudo firewall-cmd --permanent --remove-port=8080/tcp
sudo firewall-cmd --list-ports

# Rich rules (complex rules)
sudo firewall-cmd --permanent --add-rich-rule='rule family="ipv4" source address="192.168.1.0/24" service
name="ssh" accept'
sudo firewall-cmd --permanent --add-rich-rule='rule family="ipv4" source address="10.0.0.5" drop'
sudo firewall-cmd --permanent --add-rich-rule='rule family="ipv4" source address="192.168.1.100" port
protocol="tcp" port="3306" accept'

# Port forwarding
sudo firewall-cmd --permanent --add-forward-port=port=80:proto=tcp:toport=8080
sudo firewall-cmd --permanent --add-forward-port=port=80:proto=tcp:toaddr=192.168.1.100:toport=80

# Masquerading (NAT)
sudo firewall-cmd --permanent --add-masquerade
sudo firewall-cmd --permanent --remove-masquerade

# Direct rules (raw iptables)
sudo firewall-cmd --direct --add-rule ipv4 filter INPUT 0 -p tcp --dport 9000 -j ACCEPT

# ICMP
sudo firewall-cmd --permanent --add-icmp-block=echo-reply
sudo firewall-cmd --permanent --remove-icmp-block=echo-reply

# Reload
sudo firewall-cmd --reload

# Runtime vs permanent
# Without --permanent: applies immediately but lost on reload
# With --permanent: saved but requires --reload to take effect
```

18. `ssh` — Secure Shell

Syntax

```
ssh [OPTIONS] [USER@]HOST [COMMAND]
```

All Major Flags

Flag	Description
`-p PORT`	Connect to port (default: 22)
`-l USER`	Login as user
`-i KEYFILE`	Identity (private key) file
`-F CONFIG`	Config file (default: `~/.ssh/config`)
`-o OPTION`	SSH option (e.g., `-o StrictHostKeyChecking=no`)
`-v`	Verbose (use `-vv` or `-vvv` for more)
`-q`	Quiet mode
`-N`	No remote command (useful for tunnels)
`-f`	Go to background before command execution
`-T`	Disable pseudo-terminal allocation
`-t`	Force pseudo-terminal allocation
`-tt`	Force TTY even if ssh has no local TTY
`-C`	Enable compression
`-X`	Enable X11 forwarding
`-x`	Disable X11 forwarding
`-Y`	Enable trusted X11 forwarding
`-A`	Enable agent forwarding
`-a`	Disable agent forwarding
`-L [BIND:]PORT:HOST:PORT`	Local port forwarding
`-R [BIND:]PORT:HOST:PORT`	Remote port forwarding
`-D [BIND:]PORT`	Dynamic SOCKS proxy
`-J USER@HOST:PORT`	ProxyJump (jump host)
`-W HOST:PORT`	Forward stdin/stdout to HOST:PORT
`-w LOCAL:REMOTE`	TUN device forwarding
`-b ADDR`	Bind address
`-e CHAR`	Escape character (default: `~`)
`-E LOGFILE`	Append debug logs to file
`-G`	Print config after evaluating Host blocks
`-K`	Enable GSSAPI authentication and forwarding
`-k`	Disable GSSAPI forwarding
`-M`	Master mode for connection multiplexing
`-S PATH`	Control socket for multiplexing
`-O COMMAND`	Control multiplex master: `check`, `forward`, `cancel`, `exit`, `stop`
`-4`	Force IPv4
`-6`	Force IPv6
`-1`	~~Force protocol version 1~~ **Removed in OpenSSH 7.6+**
`-2`	~~Force protocol version 2~~ **Removed in OpenSSH 7.6+** (SSHv2 is now the only version)

SSH Config Options (`-o` or `~/.ssh/config`)

Option	Description
`StrictHostKeyChecking yes/no/ask`	Host key verification
`UserKnownHostsFile FILE`	Known hosts file
`PasswordAuthentication yes/no`	Allow password auth
`PubkeyAuthentication yes/no`	Allow public key auth
`IdentityFile FILE`	Private key file
`Port PORT`	Default port
`User USERNAME`	Default username
`ProxyJump HOST`	Jump host
`ProxyCommand CMD`	Proxy command
`ServerAliveInterval SEC`	Send keepalive every SEC seconds
`ServerAliveCountMax NUM`	Disconnect after NUM missed keepalives
`ConnectTimeout SEC`	Connection timeout
`Compression yes/no`	Enable compression
`ForwardAgent yes/no`	Agent forwarding
`ForwardX11 yes/no`	X11 forwarding
`ControlMaster auto/yes/no`	Connection multiplexing
`ControlPath PATH`	Multiplex socket path
`ControlPersist TIME`	Keep multiplex connection alive
`LocalForward PORT HOST:PORT`	Local port forward
`RemoteForward PORT HOST:PORT`	Remote port forward
`DynamicForward PORT`	SOCKS proxy
`AddKeysToAgent yes/no`	Auto-add keys to agent
`IdentitiesOnly yes/no`	Only use specified identity files
`LogLevel QUIET/FATAL/ERROR/INFO/VERBOSE/DEBUG`	Log verbosity
`Ciphers LIST`	Allowed ciphers
`MACs LIST`	Allowed MACs
`KexAlgorithms LIST`	Key exchange algorithms
`HostKeyAlgorithms LIST`	Host key algorithms
`BatchMode yes/no`	Non-interactive mode
`RequestTTY auto/yes/no/force`	TTY allocation
`SendEnv PATTERN`	Send environment variable
`SetEnv KEY=VALUE`	Set environment variable
`PermitLocalCommand yes/no`	Allow local command execution
`LocalCommand CMD`	Command to run locally after connect
`TCPKeepAlive yes/no`	TCP keepalive
`ExitOnForwardFailure yes/no`	Exit if forwarding fails

SSH Escape Sequences (While Connected)

Press `Enter` then the escape character (`~` by default):

Sequence	Description
`~.`	Disconnect (terminate connection)
`~^Z`	Suspend ssh
`~#`	List forwarded connections
`~&`	Background ssh (when waiting for connections to close)
`~?`	List escape sequences
`~B`	Send BREAK
`~C`	Open command line (add/remove port forwarding)
`~R`	Request rekeying
`~V`	Decrease verbosity
`~v`	Increase verbosity

Examples

```
# Basic connection
ssh user@192.168.1.50

# Connection on custom port
ssh -p 2222 user@192.168.1.50

# Use specific key
ssh -i ~/.ssh/my_key user@192.168.1.50

# Run a remote command
ssh user@192.168.1.50 "uname -a"
ssh user@192.168.1.50 "df -h; free -m"

# Local port forwarding (access remote service locally)
ssh -L 8080:localhost:80 user@remote      # localhost:8080 → remote:80
ssh -L 3306:db-server:3306 user@bastion   # localhost:3306 → db-server:3306 via bastion

# Remote port forwarding (expose local service to remote)
ssh -R 9090:localhost:3000 user@remote     # remote:9090 → localhost:3000

# SOCKS proxy (dynamic forwarding)
ssh -D 1080 user@remote                   # localhost:1080 SOCKS proxy
# Then configure browser to use SOCKS5 proxy at localhost:1080

# Jump host (ProxyJump)
ssh -J bastion@jump.example.com user@internal-server
ssh -J jump1,jump2 user@internal           # Multiple jumps

# X11 forwarding (run GUI apps remotely)
ssh -X user@remote
# Then run: firefox, gedit, etc.

# Background tunnel
ssh -fNL 8080:localhost:80 user@remote     # Tunnel in background

# Connection multiplexing
ssh -M -S /tmp/ssh_mux user@remote         # Create master
ssh -S /tmp/ssh_mux user@remote            # Reuse connection

# Copy SSH key to remote
ssh-copy-id user@192.168.1.50
ssh-copy-id -i ~/.ssh/my_key.pub user@192.168.1.50

# Generate SSH key pair
ssh-keygen -t ed25519 -C "my-key"
ssh-keygen -t rsa -b 4096 -C "my-key"

# Debug connection issues
ssh -vvv user@192.168.1.50

# Force password authentication
ssh -o PubkeyAuthentication=no user@192.168.1.50

# Disable host key checking (dangerous, use for scripts)
ssh -o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null user@host

# Mount remote filesystem
sshfs user@remote:/path /local/mountpoint

# Keep connection alive
ssh -o ServerAliveInterval=60 -o ServerAliveCountMax=3 user@remote
```

SSH Config Example (`~/.ssh/config`)

```
Host bastion
    HostName jump.example.com
    User admin
    Port 22
    IdentityFile ~/.ssh/bastion_key

Host internal
    HostName 10.0.0.50
    User deploy
    ProxyJump bastion
    IdentityFile ~/.ssh/internal_key

Host *
    ServerAliveInterval 60
    ServerAliveCountMax 3
    AddKeysToAgent yes
    IdentitiesOnly yes
    Compression yes
```

19. `scp` — Secure Copy

Syntax

```
scp [OPTIONS] SOURCE DESTINATION
```

All Flags

Flag	Description
`-r`	Recursive (copy directories)
`-P PORT`	Connect to port (note: uppercase P, unlike ssh)
`-i KEYFILE`	Identity file
`-p`	Preserve modification times, access times, modes
`-q`	Quiet (no progress bar)
`-v`	Verbose
`-C`	Enable compression
`-l LIMIT`	Limit bandwidth (Kbit/s)
`-o OPTION`	SSH option
`-F CONFIG`	SSH config file
`-S PROGRAM`	Use program for encrypted connection
`-c CIPHER`	Select cipher
`-3`	Route through local host (for copying between two remotes)
`-4`	Force IPv4
`-6`	Force IPv6
`-B`	Batch mode (no password prompt)
`-J DEST`	ProxyJump
`-O`	Use original SCP protocol (not SFTP)
`-D`	Use SFTP protocol (default on newer versions)
`-T`	Disable strict filename checking

Examples

```
# Copy local file to remote
scp file.txt user@remote:/home/user/

# Copy remote file to local
scp user@remote:/var/log/syslog ./

# Copy directory recursively
scp -r ./mydir user@remote:/home/user/

# Custom port
scp -P 2222 file.txt user@remote:/tmp/

# With specific key
scp -i ~/.ssh/my_key file.txt user@remote:/tmp/

# Preserve timestamps and permissions
scp -p file.txt user@remote:/tmp/

# Limit bandwidth to 500 Kbit/s
scp -l 500 largefile.iso user@remote:/tmp/

# Copy between two remote hosts through local
scp -3 user1@host1:/file user2@host2:/file

# Via jump host
scp -J bastion file.txt user@internal:/tmp/

# With compression
scp -C largefile.txt user@remote:/tmp/
```

20. `sftp` — Secure FTP

Interactive secure file transfer over SSH.

Syntax

```
sftp [OPTIONS] [USER@]HOST[:PATH]
```

Flags

Flag	Description
`-P PORT`	Port
`-i KEYFILE`	Identity file
`-b BATCHFILE`	Batch mode (read commands from file)
`-B BUFFERSIZE`	Buffer size for transfers
`-l LIMIT`	Bandwidth limit (Kbit/s)
`-o OPTION`	SSH option
`-F CONFIG`	SSH config file
`-r`	Recursive (for `put` / `get`)
`-R NUM`	Number of outstanding requests
`-s SUBSYSTEM`	SSH2 subsystem
`-S PROGRAM`	Program for connection
`-C`	Compression
`-v`	Verbose
`-4` / `-6`	Force IPv4/IPv6
`-J HOST`	ProxyJump

Interactive Commands

Command	Description
`ls [PATH]`	List remote directory
`lls [PATH]`	List local directory
`cd PATH`	Change remote directory
`lcd PATH`	Change local directory
`pwd`	Print remote working directory
`lpwd`	Print local working directory
`get FILE [LOCAL]`	Download file
`get -r DIR`	Download directory recursively
`put FILE [REMOTE]`	Upload file
`put -r DIR`	Upload directory recursively
`mget PATTERN`	Download multiple files
`mput PATTERN`	Upload multiple files
`mkdir DIR`	Create remote directory
`rmdir DIR`	Remove remote directory
`rm FILE`	Delete remote file
`rename OLD NEW`	Rename remote file
`chmod MODE FILE`	Change remote file permissions
`chown UID FILE`	Change remote file owner
`chgrp GID FILE`	Change remote file group
`ln SRC DST`	Create remote symlink
`lumask MASK`	Set local umask
`df [-h] [PATH]`	Show remote disk usage
`!COMMAND`	Execute local command
`exit` / `quit` / `bye`	Exit sftp
`help` / `?`	Show help
`progress`	Toggle progress display
`version`	Show SFTP version
`reget FILE`	Resume download
`reput FILE`	Resume upload

21. `rsync` — Remote File Synchronization

Syntax

```
rsync [OPTIONS] SOURCE DESTINATION
```

All Major Flags

Flag	Description
`-a` / `--archive`	Archive mode: `-rlptgoD` (recursive, links, perms, times, group, owner, devices)
`-r` / `--recursive`	Recursive
`-v` / `--verbose`	Verbose
`-z` / `--compress`	Compress during transfer
`-P`	Same as `--partial --progress`
`--partial`	Keep partially transferred files
`--progress`	Show transfer progress
`-n` / `--dry-run`	Simulate (show what would be done)
`--delete`	Delete files on destination not in source
`--delete-before`	Delete before transfer
`--delete-during`	Delete during transfer
`--delete-after`	Delete after transfer
`--delete-excluded`	Also delete excluded files on destination
`-e COMMAND`	Remote shell to use (e.g., `-e "ssh -p 2222"`)
`--exclude PATTERN`	Exclude pattern
`--exclude-from FILE`	Exclude patterns from file
`--include PATTERN`	Include pattern
`--include-from FILE`	Include patterns from file
`--filter RULE`	Add file-filtering rule
`-l` / `--links`	Copy symlinks as symlinks
`-L` / `--copy-links`	Transform symlinks into files
`-H` / `--hard-links`	Preserve hard links
`-p` / `--perms`	Preserve permissions
`-o` / `--owner`	Preserve owner
`-g` / `--group`	Preserve group
`-t` / `--times`	Preserve modification times
`-D`	Same as `--devices --specials`
`--devices`	Preserve device files
`--specials`	Preserve special files
`-S` / `--sparse`	Handle sparse files efficiently
`-x` / `--one-file-system`	Don't cross filesystem boundaries
`-u` / `--update`	Skip files newer on destination
`-c` / `--checksum`	Skip based on checksum instead of mod-time/size
`--size-only`	Skip based on size only
`-b` / `--backup`	Make backups
`--backup-dir DIR`	Backup directory
`--suffix SUFFIX`	Backup suffix (default: `~`)
`--bwlimit RATE`	Bandwidth limit (KB/s or with suffix: K, M, G)

Flag	Description
`--max-size SIZE`	Skip files larger than SIZE
`--min-size SIZE`	Skip files smaller than SIZE
`--existing`	Skip creating new files on receiver
`--ignore-existing`	Skip updating existing files
`--remove-source-files`	Delete source files after transfer
`--chmod PERMS`	Set permissions on destination
`--chown USER:GROUP`	Set owner/group on destination
`-i` / `--itemize-changes`	Show detailed changes
`--info FLAGS`	Fine-grained info output
`--stats`	Show transfer statistics
`-h` / `--human-readable`	Human-readable sizes
`--log-file FILE`	Log to file
`--log-file-format FMT`	Log format string
`-W` / `--whole-file`	Copy whole files (don't delta)
`--no-whole-file`	Force delta transfer
`-I` / `--ignore-times`	Don't skip files by time/size
`--list-only`	List files instead of transferring
`--timeout SEC`	I/O timeout
`--contimeout SEC`	Connection timeout
`--temp-dir DIR`	Create temp files in DIR
`--compare-dest DIR`	Compare against DIR
`--copy-dest DIR`	Copy from DIR if unchanged
`--link-dest DIR`	Hardlink from DIR if unchanged
`--compress-level NUM`	Compression level (0-9)
`--skip-compress LIST`	Skip compression for these suffixes
`-4` / `-6`	Force IPv4/IPv6
`--address ADDR`	Bind to address
`--port PORT`	Specify daemon port
`--sockopts OPTIONS`	Socket options
`--blocking-io`	Use blocking I/O
`--outbuf MODE`	Output buffering: `N`one, `L`ine, `B`lock
`--8-bit-output`	Don't escape high-bit chars
`--password-file FILE`	Read daemon password from file
`--early-input FILE`	Input to be consumed before transfer

Examples

```
# Sync directory to remote
rsync -avz ./mydir/ user@remote:/backup/mydir/

# Dry run (preview changes)
rsync -avzn ./mydir/ user@remote:/backup/mydir/

# Sync with delete (mirror)
rsync -avz --delete ./mydir/ user@remote:/backup/mydir/

# Exclude patterns
rsync -avz --exclude '*.log' --exclude '.git' ./src/ user@remote:/deploy/

# Use custom SSH port
rsync -avz -e "ssh -p 2222" ./mydir/ user@remote:/backup/

# Bandwidth limit (1MB/s)
rsync -avz --bwlimit=1M ./mydir/ user@remote:/backup/

# Show progress
rsync -avzP ./largefile.iso user@remote:/backup/

# Show itemized changes
rsync -avzi ./mydir/ user@remote:/backup/mydir/

# Only newer files
rsync -avzu ./mydir/ user@remote:/backup/mydir/

# Incremental backup with hard links
rsync -avz --link-dest=/backup/yesterday/ ./data/ /backup/today/

# Local sync
rsync -avz /source/ /destination/

# Checksum-based comparison
rsync -avzc ./mydir/ user@remote:/backup/

# Remove source files after transfer
rsync -avz --remove-source-files ./outbox/ user@remote:/inbox/

# Show statistics
rsync -avz --stats ./mydir/ user@remote:/backup/
```

22. `nc` / `netcat` — Network Swiss Army Knife

Syntax

```
nc [OPTIONS] HOST PORT
nc [OPTIONS] -l [PORT]
```

Flags (GNU/OpenBSD netcat)

Flag	Description
`-l`	Listen mode
`-p PORT`	Local port
`-s ADDR`	Source address
`-u`	UDP mode
`-v`	Verbose
`-w SEC`	Timeout
`-z`	Zero-I/O mode (scanning)
`-n`	Numeric only (no DNS)
`-k`	Keep listening after disconnect (accept multiple connections)
`-e PROG`	Execute program on connect (some versions)
`-c CMD`	Execute shell command on connect (some versions)
`-q SEC`	Quit after EOF on stdin, wait SEC
`-d`	Detach from stdin
`-i SEC`	Delay between lines sent
`-o FILE`	Hex dump traffic to file
`-r`	Randomize source/destination ports
`-C`	Send CRLF line endings
`-N`	Shutdown network after EOF on stdin
`-4` / `-6`	Force IPv4/IPv6
`-U`	Use Unix domain sockets
`-X PROTO`	Proxy protocol: `4` (SOCKS4), `5` (SOCKS5), `connect` (HTTP)
`-x PROXY:PORT`	Proxy address

Examples

```
# Test if a port is open
nc -zv 192.168.1.1 80
nc -zv 192.168.1.1 20-100              # Scan port range

# Simple TCP server
nc -l -p 4444

# Simple TCP client
nc 192.168.1.50 4444

# UDP mode
nc -u -l -p 5000                       # UDP server
nc -u 192.168.1.50 5000                # UDP client

# File transfer
nc -l -p 4444 > received.tar.gz        # Receiver
nc 192.168.1.50 4444 < file.tar.gz     # Sender

# Chat between machines
nc -l -p 5000                          # Machine A
nc 192.168.1.100 5000                  # Machine B

# HTTP request
echo -e "GET / HTTP/1.1\r\nHost: example.com\r\n\r\n" | nc example.com 80

# Banner grabbing
echo "" | nc -w 3 192.168.1.1 22       # SSH banner
echo "" | nc -w 3 192.168.1.1 25       # SMTP banner

# Port forwarding (with process substitution)
nc -l -p 8080 -c "nc 192.168.1.100 80"

# Keep listening for multiple connections
nc -lk -p 4444

# Proxy through SOCKS5
nc -X 5 -x proxy:1080 target 80

# With timeout
nc -w 5 192.168.1.1 80

# Directory transfer with tar
tar czf - ./mydir | nc 192.168.1.50 4444         # Sender
nc -l -p 4444 | tar xzf -                         # Receiver

# Backdoor shell (for testing only)
nc -l -p 4444 -e /bin/bash                         # Dangerous!
```

23. `iperf3` — Network Performance Testing

Syntax

```
iperf3 -s [OPTIONS]          # Server
iperf3 -c HOST [OPTIONS]     # Client
```

Server Flags

Flag	Description
`-s`	Run as server
`-p PORT`	Listen port (default: 5201)
`-D`	Daemon mode
`-1`	One-off: handle one client then exit
`-I FILE`	Write PID to file
`--idle-timeout SEC`	Idle timeout

Client Flags

Flag	Description
`-c HOST`	Run as client connecting to HOST
`-p PORT`	Server port (default: 5201)
`-t SEC`	Test duration (default: 10)
`-n SIZE`	Transfer this many bytes
`-b RATE`	Target bandwidth (e.g., `100M`)
`-P NUM`	Parallel streams
`-R`	Reverse mode (server sends, client receives)
`--bidir`	Bidirectional test
`-u`	UDP mode
`-l LENGTH`	Buffer/packet length
`-w SIZE`	Socket buffer/window size
`-M MSS`	TCP MSS
`-N`	Set TCP no delay
`-4` / `-6`	Force IPv4/IPv6
`-B ADDR`	Bind to address
`-i SEC`	Reporting interval
`-f FORMAT`	Format: `k` Kbits, `m` Mbits, `g` Gbits, `K` KBytes, `M` MBytes
`-J`	JSON output
`--logfile FILE`	Log to file
`-T TITLE`	Prefix output with title
`--connect-timeout MS`	Connection timeout
`-A CPU`	CPU affinity
`--get-server-output`	Get server output on client
`--udp-counters-64bit`	64-bit UDP counters
`--repeating-payload`	Use repeating pattern
`-S TOS`	Set TOS/DSCP
`--dont-fragment`	Set DF bit
`-Z`	Use zero copy
`--timestamps[=FORMAT]`	Add timestamps

Examples

```
# Server
iperf3 -s

# Basic client test
iperf3 -c 192.168.1.1

# Test for 30 seconds
iperf3 -c 192.168.1.1 -t 30

# 4 parallel streams
iperf3 -c 192.168.1.1 -P 4

# Reverse (test download speed)
iperf3 -c 192.168.1.1 -R

# Bidirectional
iperf3 -c 192.168.1.1 --bidir

# UDP test at 100 Mbps
iperf3 -c 192.168.1.1 -u -b 100M

# JSON output
iperf3 -c 192.168.1.1 -J > result.json

# Custom window size
iperf3 -c 192.168.1.1 -w 256K

# Server on custom port
iperf3 -s -p 9999
iperf3 -c 192.168.1.1 -p 9999
```

24. `ethtool` — NIC Hardware Settings

Flags

Flag	Description
(no flag)	Show NIC settings
`-i`	Show driver info
`-S`	Show NIC statistics
`-a`	Show pause parameters
`-A`	Change pause parameters
`-c`	Show coalesce settings
`-C`	Change coalesce settings
`-g`	Show ring buffer sizes
`-G`	Change ring buffer sizes
`-k`	Show offload settings
`-K`	Change offload settings
`-l`	Show channel count
`-L`	Change channel count
`-m`	Show transceiver module info
`-n` / `--show-nfc`	Show network flow classification rules
`-p`	Identify interface (blink LED)
`-r`	Restart auto-negotiation
`-s`	Change NIC settings
`-t`	Self-test
`-T`	Show time stamping capabilities
`--show-eee`	Show EEE (Energy Efficient Ethernet)
`--set-eee`	Change EEE settings
`--show-fec`	Show FEC (Forward Error Correction)
`--set-fec`	Change FEC settings
`--show-priv-flags`	Show private flags
`--set-priv-flags`	Set private flags

Examples

```
# Show NIC settings
sudo ethtool eth0

# Show driver info
sudo ethtool -i eth0

# Show statistics
sudo ethtool -S eth0

# Set speed/duplex
sudo ethtool -s eth0 speed 1000 duplex full autoneg off

# Show offload settings
sudo ethtool -k eth0

# Disable TCP segmentation offload
sudo ethtool -K eth0 tso off

# Enable generic receive offload
sudo ethtool -K eth0 gro on

# Show ring buffer sizes
sudo ethtool -g eth0

# Increase ring buffer
sudo ethtool -G eth0 rx 4096 tx 4096

# Blink NIC LED (identify)
sudo ethtool -p eth0 5                  # Blink for 5 seconds

# Show pause frames
sudo ethtool -a eth0

# Restart auto-negotiation
sudo ethtool -r eth0

# Show channel count
sudo ethtool -l eth0

# Show coalesce settings
sudo ethtool -c eth0

# Run self-test
sudo ethtool -t eth0
```

25. `nmcli` — NetworkManager CLI

Main Commands

```
# General status
nmcli general status
nmcli general hostname
nmcli general permissions
nmcli general logging

# Networking
nmcli networking on
nmcli networking off
nmcli networking connectivity

# Device management
nmcli device status
nmcli device show
nmcli device show eth0
nmcli device connect eth0
nmcli device disconnect eth0
nmcli device wifi list
nmcli device wifi rescan
nmcli device wifi connect "SSID" password "PASS"
nmcli device wifi hotspot ssid "MyHotspot" password "pass1234"

# Connection management
nmcli connection show
nmcli connection show --active
nmcli connection show "My Connection"
nmcli connection up "My Connection"
nmcli connection down "My Connection"
nmcli connection delete "My Connection"
nmcli connection reload
nmcli connection load /etc/NetworkManager/system-connections/myconn.nmconnection

# Create connections
nmcli connection add type ethernet con-name "static" ifname eth0 \
  ipv4.addresses 192.168.1.100/24 \
  ipv4.gateway 192.168.1.1 \
  ipv4.dns "8.8.8.8 8.8.4.4" \
  ipv4.method manual

nmcli connection add type wifi con-name "home-wifi" ifname wlan0 \
  ssid "MyNetwork" wifi-sec.key-mgmt wpa-psk wifi-sec.psk "password"

nmcli connection add type bond con-name "bond0" ifname bond0 \
  bond.options "mode=802.3ad,miimon=100"

nmcli connection add type bridge con-name "br0" ifname br0

nmcli connection add type vlan con-name "vlan100" ifname eth0.100 \
  dev eth0 id 100

# Modify connections
nmcli connection modify "My Connection" ipv4.dns "1.1.1.1"
nmcli connection modify "My Connection" +ipv4.dns "8.8.8.8"
nmcli connection modify "My Connection" ipv4.addresses "192.168.1.200/24"
nmcli connection modify "My Connection" connection.autoconnect yes

# Monitor
nmcli monitor
```

26–30: Quick Reference for Remaining Commands

`whois`

```
whois example.com                    # Domain lookup
whois 8.8.8.8                        # IP lookup
whois -h whois.arin.net 8.8.8.8     # Query specific WHOIS server
```

`hostname` / `hostnamectl`

```
hostname                              # Show hostname
hostname -f                           # FQDN
hostname -I                           # All IPs
hostname -i                           # Loopback address
hostname -d                           # Domain name
hostname -s                           # Short hostname
sudo hostnamectl set-hostname newname  # Set hostname permanently
hostnamectl status                     # Show full hostname info
```

`ipcalc`

```
ipcalc 192.168.1.100/24              # Calculate network info
ipcalc 192.168.1.100 255.255.255.0   # With netmask
ipcalc 10.0.0.0/8 -s 200 100 50     # Subnet splitting
```

`lsof` (Network)

```
sudo lsof -i                         # All network connections
sudo lsof -i :80                     # Connections on port 80
sudo lsof -i tcp                     # TCP connections only
sudo lsof -i udp                     # UDP connections only
sudo lsof -i @192.168.1.50           # Connections to host
sudo lsof -i :1-1024                 # Privileged ports
sudo lsof -i -P -n                   # Numeric, no resolve
sudo lsof -iTCP -sTCP:LISTEN         # Only listening TCP
sudo lsof -iTCP -sTCP:ESTABLISHED    # Only established TCP
sudo lsof -u username -i             # User's network connections
```

`iftop`

```
sudo iftop                            # Default interface
sudo iftop -i eth0                    # Specific interface
sudo iftop -n                         # Numeric (no DNS)
sudo iftop -N                         # No port resolution
sudo iftop -P                         # Show ports
sudo iftop -B                         # Bytes instead of bits
sudo iftop -f "dst port 80"          # BPF filter
sudo iftop -F 192.168.1.0/24         # Filter by network
```

`vnstat`

```
vnstat                                  # Summary for default interface
vnstat -i eth0                          # Specific interface
vnstat -h                               # Hourly stats
vnstat -hg                              # Hourly graph
vnstat -d                               # Daily stats
vnstat -m                               # Monthly stats
vnstat -y                               # Yearly stats
vnstat -t                               # Top 10 days
vnstat -l                               # Live monitoring
vnstat -5                               # 5-minute stats
vnstat --json                           # JSON output
vnstat --add -i eth0                    # Add interface to monitor
vnstat --remove -i eth0                 # Remove interface
```

`nethogs`

```
sudo nethogs                            # All interfaces
sudo nethogs eth0                       # Specific interface
sudo nethogs -d 5                       # Refresh every 5 seconds
sudo nethogs -t                         # Tracemode (non-interactive)
sudo nethogs -v 3                       # Show both sent/received
```

`tc` (Traffic Control) — Key Commands

```
# Show current qdisc
tc qdisc show dev eth0

# Add delay
sudo tc qdisc add dev eth0 root netem delay 100ms

# Add delay with jitter
sudo tc qdisc add dev eth0 root netem delay 100ms 20ms

# Add packet loss
sudo tc qdisc add dev eth0 root netem loss 5%

# Add bandwidth limit
sudo tc qdisc add dev eth0 root tbf rate 1mbit burst 32kbit latency 400ms

# Combined
sudo tc qdisc add dev eth0 root netem delay 50ms loss 1% rate 10mbit

# Remove rules
sudo tc qdisc del dev eth0 root

# Show filters
tc filter show dev eth0
```

`iw` (Wireless)

```
iw dev                                # List wireless devices
iw dev wlan0 info                     # Interface info
iw dev wlan0 scan                     # Scan for networks
iw dev wlan0 link                     # Connection info
iw dev wlan0 station dump             # Station statistics
iw phy phy0 info                      # Physical device capabilities
sudo iw dev wlan0 connect "SSID"      # Connect (open network)
sudo iw dev wlan0 disconnect          # Disconnect
sudo iw dev wlan0 set power_save off  # Disable power save
iw reg get                            # Regulatory domain
sudo iw reg set US                    # Set regulatory domain
```

`socat` — Multipurpose Relay

```
# TCP port forwarder
socat TCP-LISTEN:8080,fork TCP:192.168.1.100:80

# UDP forwarder
socat UDP-LISTEN:5000,fork UDP:192.168.1.100:5000

# Simple HTTP server
socat TCP-LISTEN:8080,fork EXEC:"echo HTTP/1.0 200 OK; echo; cat index.html"

# TLS server
socat OPENSSL-LISTEN:443,cert=server.pem,fork TCP:localhost:80

# Unix socket to TCP
socat UNIX-LISTEN:/tmp/my.sock,fork TCP:192.168.1.100:3306

# Two-way pipe between processes
socat EXEC:"/bin/cat" EXEC:"/bin/cat"

# Serial port to TCP
socat TCP-LISTEN:2000,fork /dev/ttyUSB0,b9600
```

`openssl s_client` — SSL/TLS Testing

```
# Test TLS connection
openssl s_client -connect example.com:443

# Show certificate chain
openssl s_client -connect example.com:443 -showcerts

# Test specific TLS version
openssl s_client -connect example.com:443 -tls1_2
openssl s_client -connect example.com:443 -tls1_3

# Test with SNI
openssl s_client -connect example.com:443 -servername example.com

# Check certificate dates
echo | openssl s_client -connect example.com:443 2>/dev/null | openssl x509 -noout -dates

# Check certificate subject
echo | openssl s_client -connect example.com:443 2>/dev/null | openssl x509 -noout -subject

# STARTTLS for SMTP
openssl s_client -connect mail.example.com:25 -starttls smtp

# STARTTLS for IMAP
openssl s_client -connect mail.example.com:143 -starttls imap

# Check supported ciphers
openssl s_client -connect example.com:443 -cipher 'ALL'

# Verify certificate
openssl s_client -connect example.com:443 -verify 5 -CApath /etc/ssl/certs/
```

`telnet` (Connectivity Testing)

```
# Test TCP port
telnet 192.168.1.1 80
telnet 192.168.1.1 22
telnet 192.168.1.1 25

# HTTP request
telnet example.com 80
# Then type:
# GET / HTTP/1.1
# Host: example.com
# (blank line)
```

`dhclient` — DHCP Client

```
sudo dhclient eth0                    # Request DHCP lease
sudo dhclient -r eth0                 # Release DHCP lease
sudo dhclient -v eth0                 # Verbose
sudo dhclient -6 eth0                 # DHCPv6
sudo dhclient -d eth0                 # Debug (foreground)
```

`resolvectl` / `systemd-resolve`

```
resolvectl status                      # DNS status
resolvectl query example.com           # Resolve name
resolvectl dns eth0 8.8.8.8 8.8.4.4  # Set DNS for interface
resolvectl domain eth0 example.com     # Set search domain
resolvectl flush-caches                # Flush DNS cache
resolvectl statistics                  # Cache statistics
```

`tshark` — Terminal Wireshark

```
# Capture on interface
tshark -i eth0

# Capture with filter
tshark -i eth0 -f "tcp port 80"

# Read from pcap
tshark -r capture.pcap

# Display filter
tshark -i eth0 -Y "http.request"

# Save to file
tshark -i eth0 -w output.pcap

# Show specific fields
tshark -i eth0 -T fields -e ip.src -e ip.dst -e tcp.port

# Packet count
tshark -i eth0 -c 100

# Verbose decode
tshark -i eth0 -V

# Statistics
tshark -i eth0 -z conv,tcp
tshark -i eth0 -z io,stat,1
tshark -r capture.pcap -z endpoints,ip
```

`ab` — Apache HTTP Benchmarking

```
# 1000 requests, 10 concurrent
ab -n 1000 -c 10 http://example.com/

# With POST data
ab -n 100 -c 5 -p data.json -T "application/json" http://example.com/api

# With custom header
ab -n 100 -c 10 -H "Authorization: Bearer TOKEN" http://example.com/

# Keep-alive connections
ab -n 1000 -c 10 -k http://example.com/

# With timeout
ab -n 100 -c 10 -s 30 http://example.com/
```

`fuser` — Identify Processes Using Ports

```
# Find process on port 80
sudo fuser 80/tcp

# Verbose
sudo fuser -v 80/tcp

# Kill process on port 80
sudo fuser -k 80/tcp

# Find process on UDP port 53
sudo fuser 53/udp

# Named socket
sudo fuser -v /var/run/docker.sock
```

`bridge` — Bridge Management

```
# Show bridge info
bridge link show
bridge fdb show
bridge vlan show
bridge mdb show

# Add FDB entry
sudo bridge fdb add 00:11:22:33:44:55 dev eth0 master

# Show STP info
bridge link show dev br0
```

System Network Configuration Files

File	Purpose
`/etc/hostname`	System hostname
`/etc/hosts`	Static hostname-to-IP mapping
`/etc/resolv.conf`	DNS resolver configuration (nameserver, search domains)
`/etc/nsswitch.conf`	Name service switch (order of name resolution)
`/etc/network/interfaces`	Debian/Ubuntu network config (traditional)
`/etc/netplan/*.yaml`	Ubuntu Netplan config (modern)
`/etc/sysconfig/network-scripts/ifcfg-*`	RHEL/CentOS network config
`/etc/NetworkManager/`	NetworkManager config directory
`/etc/systemd/network/*.network`	systemd-networkd config
`/etc/iptables/rules.v4`	Saved iptables IPv4 rules
`/etc/iptables/rules.v6`	Saved iptables IPv6 rules
`/proc/sys/net/ipv4/ip_forward`	IPv4 forwarding flag
`/proc/sys/net/ipv6/conf/all/forwarding`	IPv6 forwarding flag
`/etc/sysctl.conf`	Kernel parameter configuration
`/etc/ssh/sshd_config`	SSH server configuration
`/etc/ssh/ssh_config`	SSH client configuration
`~/.ssh/config`	Per-user SSH client configuration
`~/.ssh/known_hosts`	Known SSH host keys
`~/.ssh/authorized_keys`	Authorized public keys for SSH login

Kernel Network Parameters (sysctl)

```
# Enable IP forwarding
sudo sysctl -w net.ipv4.ip_forward=1
sudo sysctl -w net.ipv6.conf.all.forwarding=1

# Disable ICMP redirects
sudo sysctl -w net.ipv4.conf.all.accept_redirects=0
sudo sysctl -w net.ipv4.conf.all.send_redirects=0

# Enable SYN cookies (DDoS protection)
sudo sysctl -w net.ipv4.tcp_syncookies=1

# Increase connection tracking table
sudo sysctl -w net.netfilter.nf_conntrack_max=262144

# Increase socket buffer sizes
sudo sysctl -w net.core.rmem_max=16777216
sudo sysctl -w net.core.wmem_max=16777216

# TCP tuning
sudo sysctl -w net.ipv4.tcp_window_scaling=1
sudo sysctl -w net.ipv4.tcp_timestamps=1
sudo sysctl -w net.ipv4.tcp_sack=1
sudo sysctl -w net.ipv4.tcp_fin_timeout=30
sudo sysctl -w net.ipv4.tcp_keepalive_time=600
sudo sysctl -w net.ipv4.tcp_max_syn_backlog=4096

# Increase port range
sudo sysctl -w net.ipv4.ip_local_port_range="1024 65535"

# Disable IPv6 (if not needed)
sudo sysctl -w net.ipv6.conf.all.disable_ipv6=1

# Prevent IP spoofing
sudo sysctl -w net.ipv4.conf.all.rp_filter=1

# Don't accept source routing
sudo sysctl -w net.ipv4.conf.all.accept_source_route=0

# Log martian packets
sudo sysctl -w net.ipv4.conf.all.log_martians=1

# Make permanent
echo "net.ipv4.ip_forward = 1" | sudo tee -a /etc/sysctl.conf
sudo sysctl -p                        # Reload
```

End of Complete Linux Network Commands Reference